Notes on

Mobile App Development

Contents

CH -1 Dart Introduction

Dart Programming Language

- • The Dart programming language is a language developed by Google.

▪It is widely used at Google and can be used for:

- • mobile app development using Flutter
- • web development (using Dart2JS)

server side (using various libraries and frameworks)

- • The syntax of Dart is very close to Java and C++
- • Executing Script Online with DartPad •

IDE For Dart

Visual Studio Code

Android Studio

IntelliJ IDEA

WebStorm (and other JetBrains IDEs)

Eclipse

Version

Manual Installation Download From Official Site

Stable channel

#

Stable channel builds are tested and approved for production use.
Version:3.7.33.7.23.7.13.7.03.6.23.6.13.6.03.5.43.5.33.5.23.5.13.5.03.4.43.4.33.4.23.4.13.4.03.3.

43.3.33.3.23.3.13.3.03.2.63.2.53.2.43.2.33.2.23.2.13.2.03.1.53.1.43.1.33.1.23.1.13.1.03.0.73.0.63

.0.53.0.43.0.33.0.23.0.13.0.02.19.62.19.52.19.42.19.32.19.22.19.12.19.02.18.72.18.62.18.52.18.4

2.18.32.18.22.18.12.18.02.17.72.17.62.17.52.17.32.17.12.17.02.16.22.16.12.16.02.15.12.15.02.1

4.42.14.32.14.22.14.12.14.02.13.42.13.32.13.12.13.02.12.42.12.32.12.22.12.12.12.02.10.52.10.4

2.10.32.10.22.10.12.10.02.9.32.9.22.9.12.9.02.8.42.8.32.8.22.8.12.7.22.7.12.7.02.6.12.6.02.5.22.

5.12.5.02.4.12.4.02.3.22.3.12.3.02.2.02.1.12.1.02.0.01.24.31.24.21.24.11.24.01.23.01.22.11.22.0

1.21.11.21.01.20.11.19.11.19.01.18.11.18.01.17.11.17.01.16.11.16.01.15.01.14.21.14.11.14.01.13

.21.13.11.13.01.12.21.12.11.12.01.11.31.11.11.11.01.10.11.10.01.9.31.9.11.8.51.8.31.8.01.7.21.6.

01.5.81.5.31.5.21.5.11.4.31.4.21.4.01.3.61.3.31.3.01.2.01.1.31.1.11.0.0-rev.10.307981.0.0-rev.3.

301880.8.10-rev.10.301070.8.10-rev.6.300360.8.10-rev.3.29803

OS:All macOS Linux Windows:-

Version	OS	Architecture	Release date	Downloads
3.7.3 (ref 633eb6b)	Windows	x64	Apr 17, 2025	Dart SDK (SHA-256)
3.7.3 (ref 633eb6b)	Windows	IA32	Apr 17, 2025	Dart SDK (SHA-256)
3.7.3 (ref 633eb6b)	Windows	ARM64	Apr 17, 2025	Dart SDK (SHA-256)
3.7.3 (ref 633eb6b)	---	---	Apr 17, 2025	API Docs

Beta channel

#

Beta channel builds are preview builds for the stable channel. We recommend testing, but not releasing, your apps against beta to preview new features or test compatibility with future releases. Beta channel builds are not suitable for production use.

Version:3.8.0-278.2.beta3.8.0-278.1.beta3.8.0-171.2.beta3.8.0-171.1.beta3.8.0-70.1.beta3.7.0-32

3.3.beta3.7.0-323.2.beta3.7.0-323.1.beta3.7.0-209.1.beta3.6.03.6.0-334.5.beta3.6.0-334.4.beta3.6

.0-334.3.beta3.6.0-334.2.beta3.6.0-216.1.beta3.6.0-149.3.beta3.5.13.5.03.5.0-323.2.beta3.5.0-32

3.1.beta3.5.0-180.3.beta3.5.0-180.2.beta3.4.23.4.13.4.03.4.0-282.4.beta3.4.0-282.3.beta3.4.0-282

.2.beta3.4.0-282.1.beta3.4.0-190.2.beta3.4.0-190.1.beta3.4.0-99.1.beta3.3.03.3.0-279.3.beta3.3.0-

279.2.beta3.3.0-279.1.beta3.3.0-174.3.beta3.3.0-174.2.beta3.3.0-91.1.beta3.2.23.2.13.2.03.2.0-21

0.4.beta3.2.0-210.3.beta3.2.0-210.2.beta3.2.0-210.1.beta3.2.0-134.1.beta3.2.0-42.2.beta3.2.0-42.

beta3.1.03.1.0-262.3.beta3.1.0-262.2.beta3.1.0-262.1.beta3.1.0-163.1.beta3.1.0-63.1.beta3.0.0-

417.4.beta3.0.0-417.3.beta3.0.0-417.2.beta3.0.0-417.1.beta3.0.0-290.3.beta3.0.0-290.2.beta3.0.0-

290.1.beta3.0.0-218.1.beta2.19.22.19.0-444.6.beta2.19.0-444.4.beta2.19.0-444.3.beta2.19.0-444.

beta2.19.0-444.1.beta2.19.0-374.2.beta2.19.0-374.1.beta2.19.0-255.2.beta2.19.0-146.2.beta2.1

9.0-146.1.beta2.18.02.18.0-271.8.beta2.18.0-271.7.beta2.18.0-271.4.beta2.18.0-271.2.beta2.18.0

-165.1.beta2.18.0-44.1.beta2.17.12.17.02.17.0-266.10.beta2.17.0-266.8.beta2.17.0-266.7.beta2.1
7.0-266.5.beta2.17.0-266.1.beta2.17.0-182.2.beta2.17.0-182.1.beta2.17.0-69.2.beta2.17.0-69.1.b

eta2.16.12.16.02.16.0-134.6.beta2.16.0-134.5.beta2.16.0-134.1.beta2.16.0-80.1.beta2.15.02.15.0-

268.18.beta2.15.0-268.8.beta2.15.0-178.1.beta2.15.0-82.2.beta2.15.0-82.1.beta2.14.12.14.0-377.

8.beta2.14.0-377.7.beta2.14.0-377.4.beta2.14.0-377.1.beta2.14.0-301.2.beta2.14.0-188.5.beta2.1

4.0-188.3.beta2.14.0-188.1.beta2.13.32.13.12.13.02.13.0-211.14.beta2.13.0-211.13.beta2.13.0-2

11.6.beta2.13.0-211.1.beta2.13.0-116.1.beta2.12.12.12.02.12.0-259.16.beta2.12.0-259.15.beta2.1

2.0-259.14.beta2.12.0-259.12.beta2.12.0-259.9.beta2.12.0-259.8.beta2.12.0-259.1.beta2.12.0-13

3.7.beta2.12.0-133.6.beta2.12.0-133.2.beta2.12.0-133.1.beta2.12.0-29.10.beta2.12.0-29.7.beta2.1

1.0-213.5.beta2.11.0-213.4.beta2.11.0-213.1.beta2.10.0-110.5.beta2.10.0-110.3.beta2.10.0-110.1.

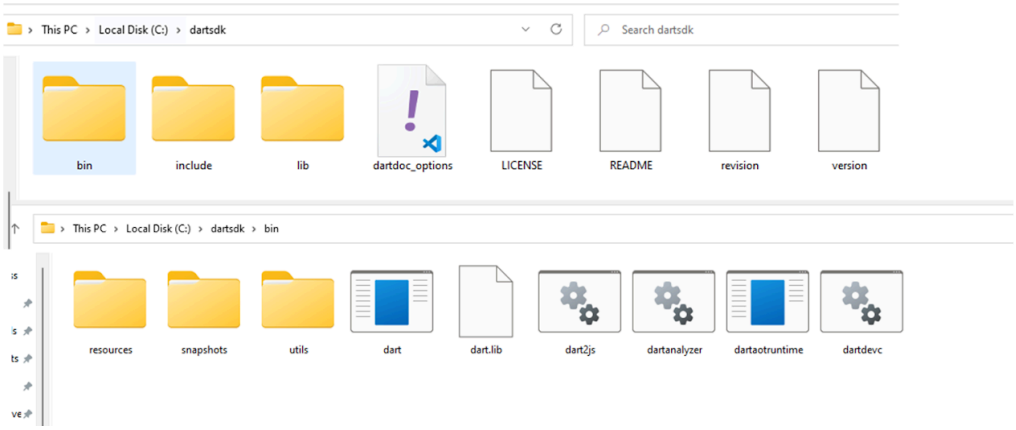
beta2.10.0-7.3.beta2.10.0-7.2.beta2.10.0-7.1.beta2.9.02.9.0-21.10.beta2.9.0-21.4.beta2.9.0-21.2.b

eta2.9.0-21.1.beta2.9.0-14.1.beta2.9.0-8.2.beta2.9.0-8.1.beta2.8.12.8.0-20.11.beta

OS:All macOS Linux Windows

Version	OS	Architecture	Release date	Downloads
3.8.0-278.2.beta (ref 9003f79)	Windows	x64	Apr 16, 2025	Dart SDK (SHA-256)
3.8.0-278.2.beta (ref 9003f79)	Windows	ARM64	Apr 16, 2025	Dart SDK (SHA-256)
3.8.0-278.2.beta (ref 9003f79)	---	---	Apr 16, 2025	API Docs

Dart SDK Setup



- •Create New Folder named dart-sdk in C or Any Drive.
- •Extract all Files in same folder same as below image

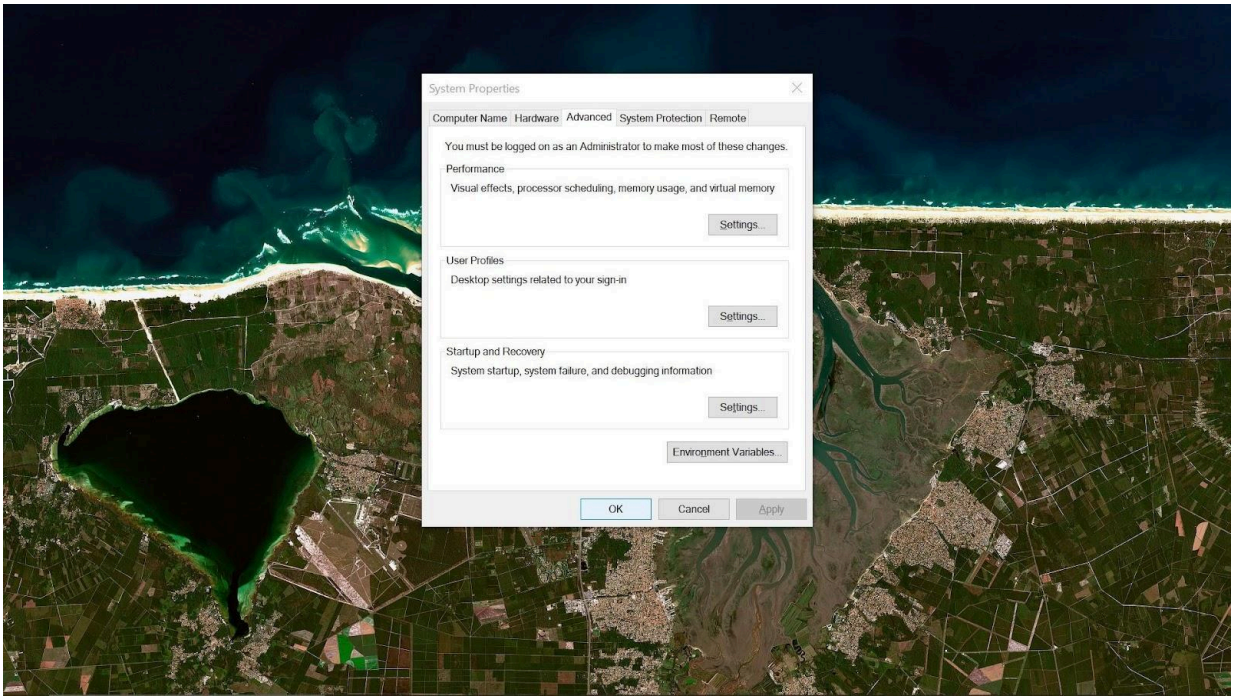
Add Dart Path to PATH Environment Variable

Open Environment Variables. Under System variables, click on Path and click Edit button.

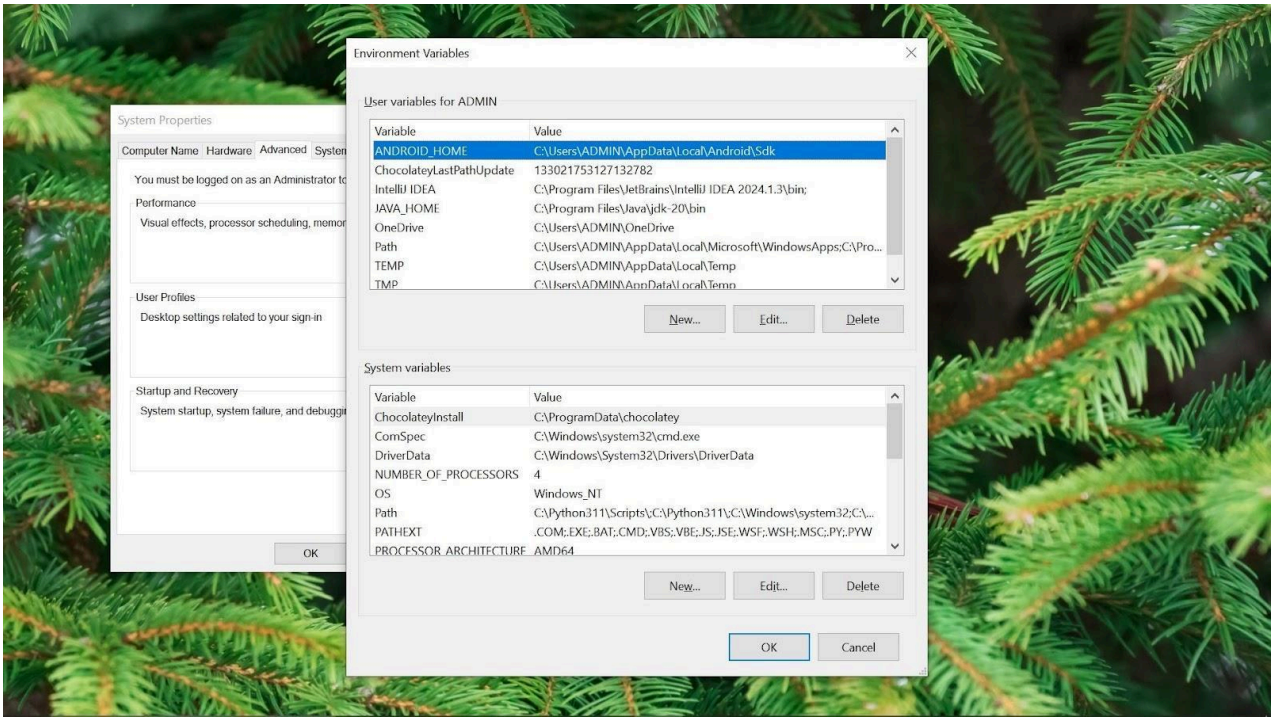
Edit environment variable window appears. Click on New and paste the dart sdk bin path as shown below.

C:\dartsdk\bin

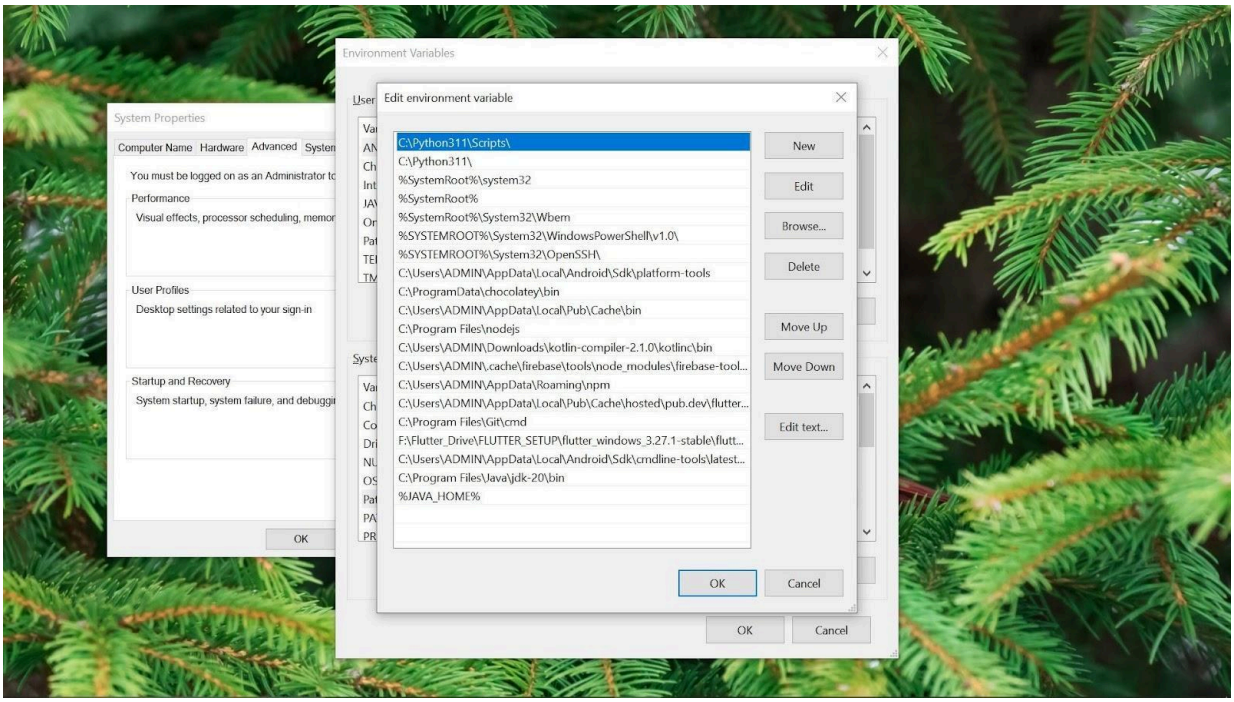
Step 1 : Click on search box and find Edit the System environment variables



Step 2 : Click on Environment Variables button

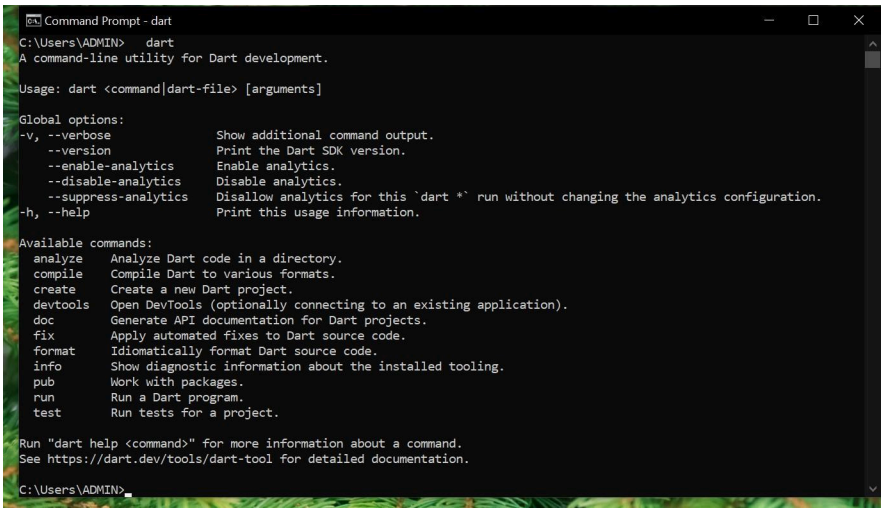


Step 3 : Then Search Path in system variable and double click on it

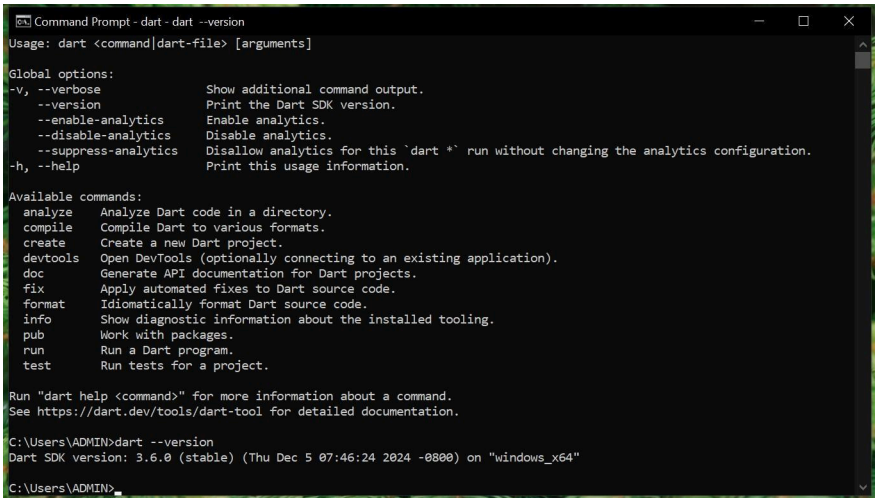


Step 4 : Insert your path here / or click on new and add path here

Check Dart in command Prompt

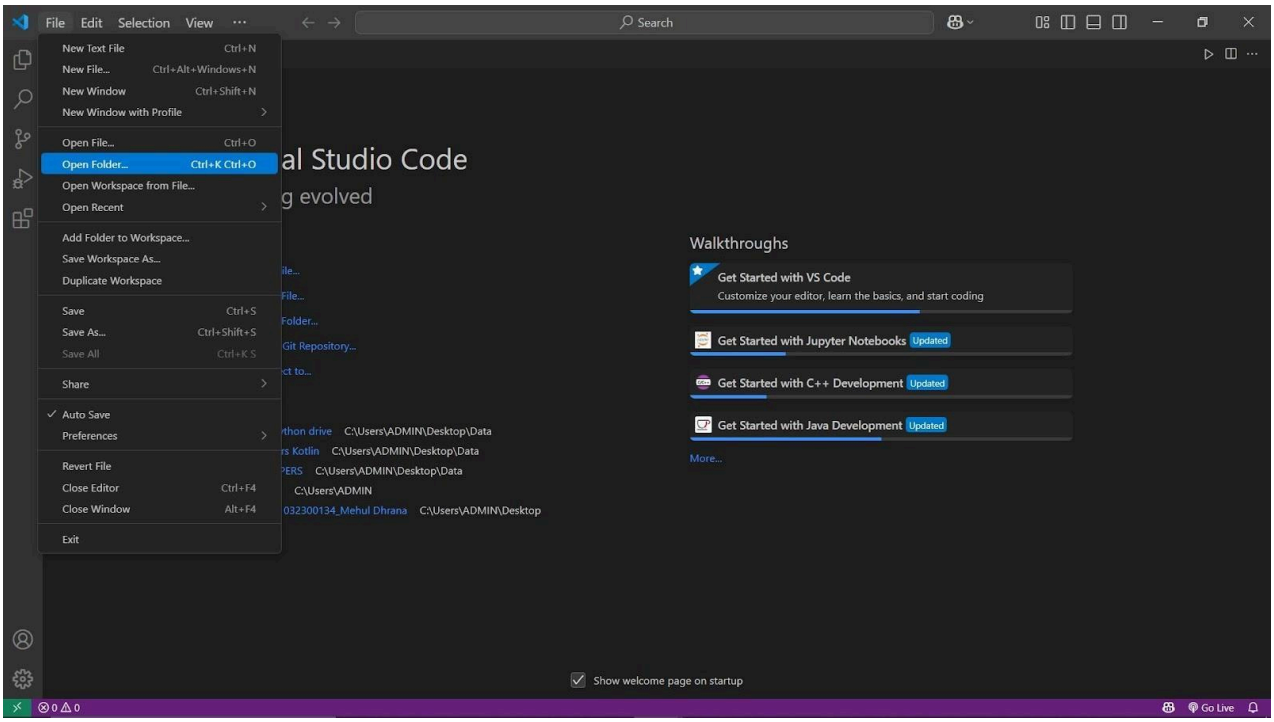


Version checking

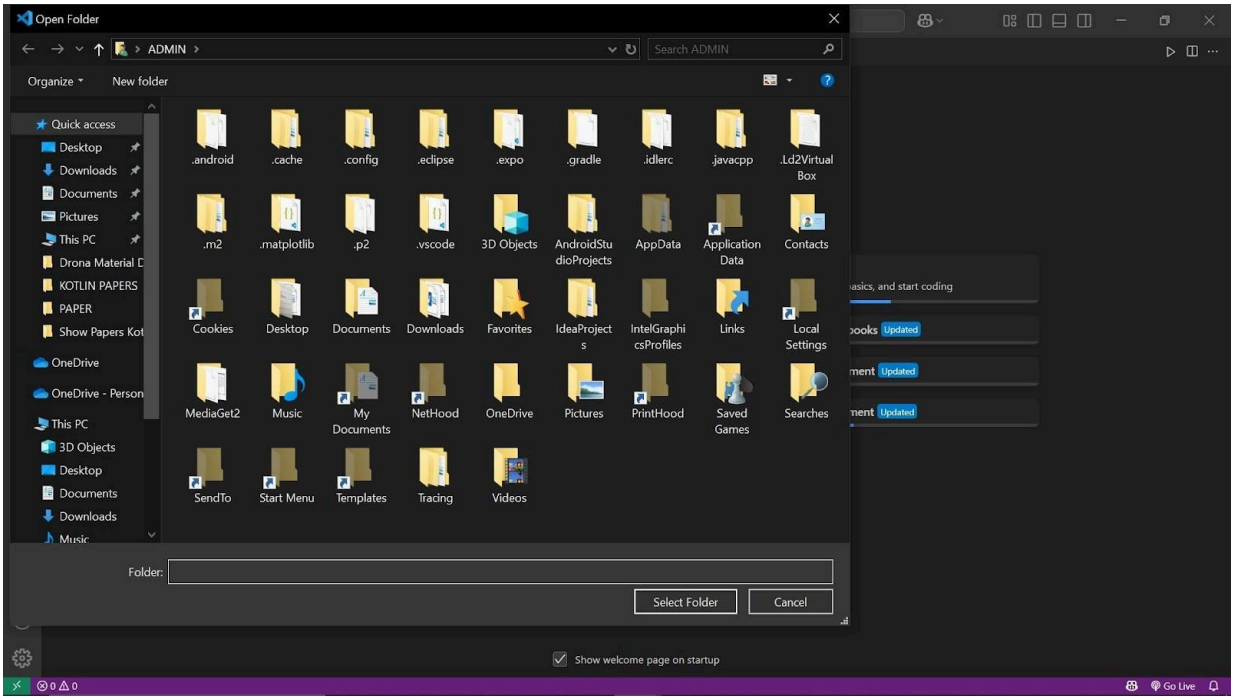


Folder Setup

Create new folder and open in Visual Studio Code Step 1 : open the folder

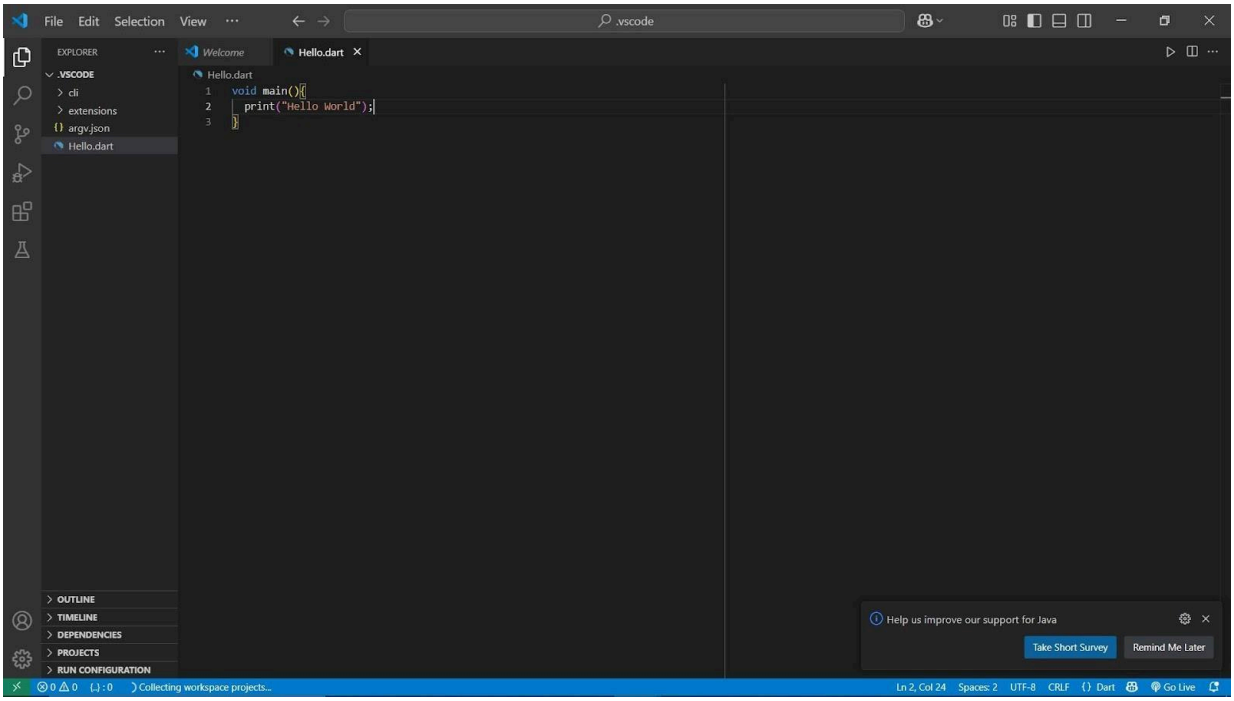


Step 2 : Select the folder

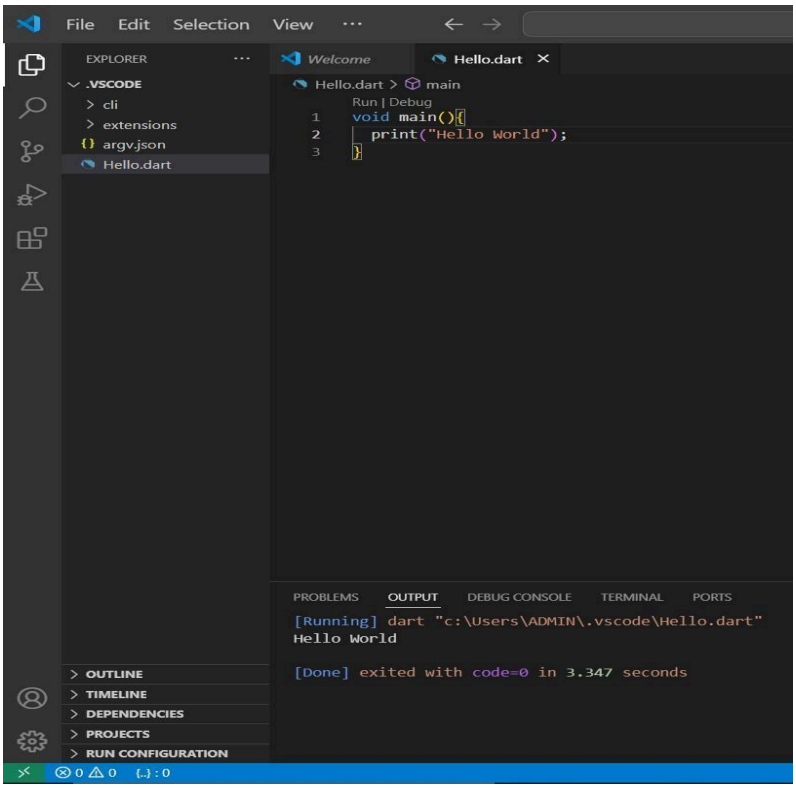


Step 3 : Create dart file

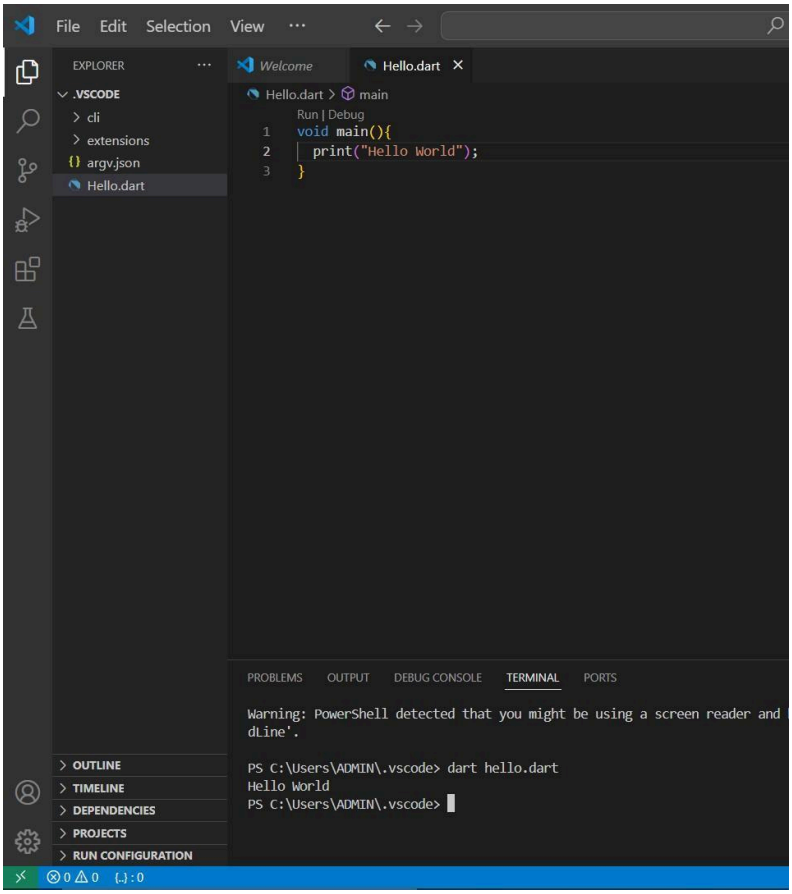
Step 4 : Write a simple program



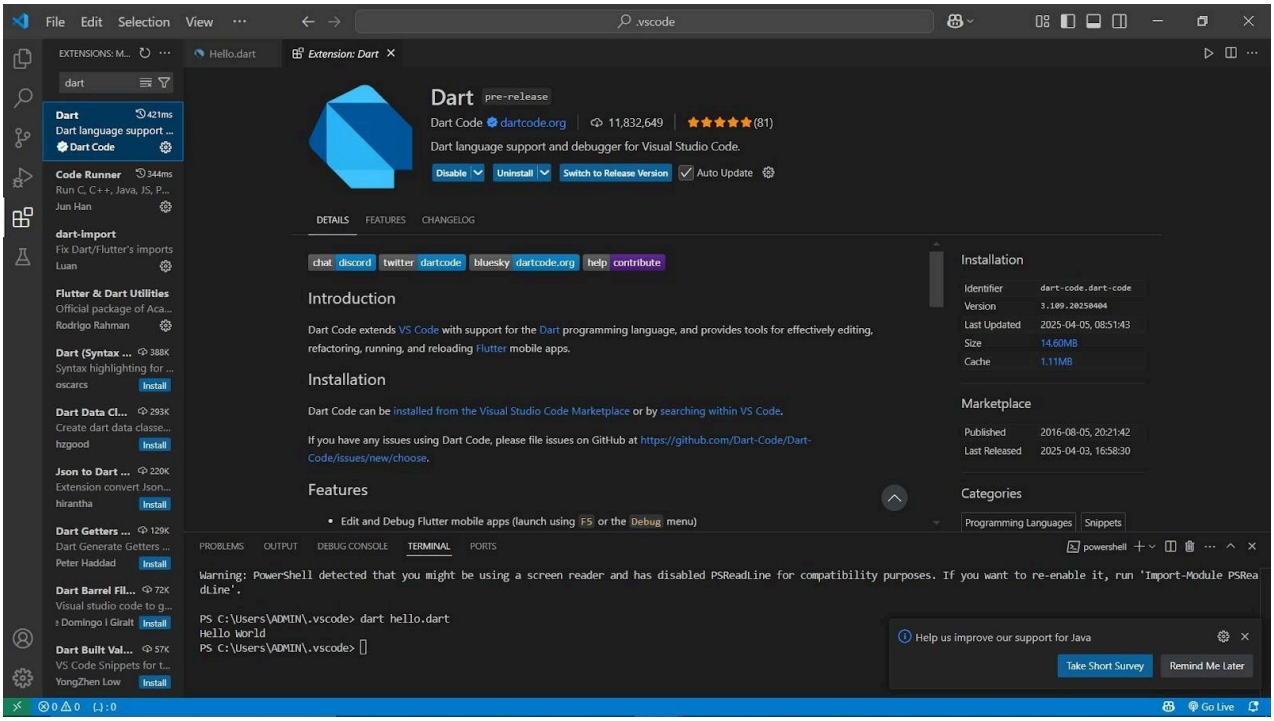
Step 4 : Execute the program



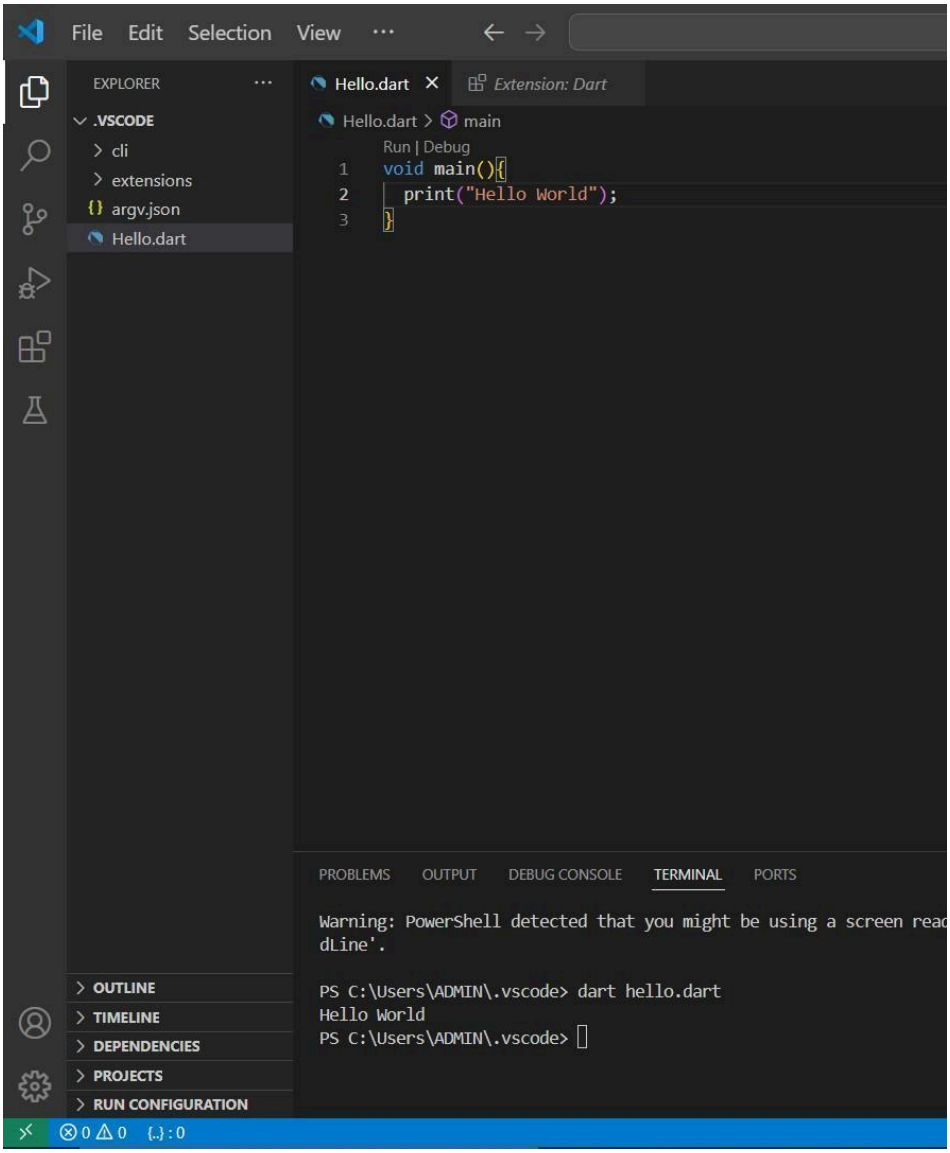
Execute program using command



Download Dart Extension



First Dart Program



Execute a Dart Program via Terminal

- •Create New Folder in D Drive named “dartdemo”
- •Create New file hello.dart
- •Navigate to the path of the current project

Type the following command in the Terminal window

- •dart file_name.dart

Explanation

- •The main() function is a predefined method in Dart. This method acts as the entry point to the application.
- •print() is a predefined function that prints the specified string or value to the standard output

Dart Comments

- •Comments are statements that are ignored by Dart compiler.

You can write comments any where in the program between dart statements.

- •three different types of comments supported by Dart programming language.

Single Line Comments

- •Block (Multi-line) Comments

▪Documentation Comments

Single Line Comments

- •To write single line comments, use double forward slash // and then write your comment after that in the same line.

void main(){

//this is comment //this is another comment

}

Block (Multi-Line) Comments

You can write multiple line comments enclosed between /* and */. Following is an example of how to write block comments.

void main(){

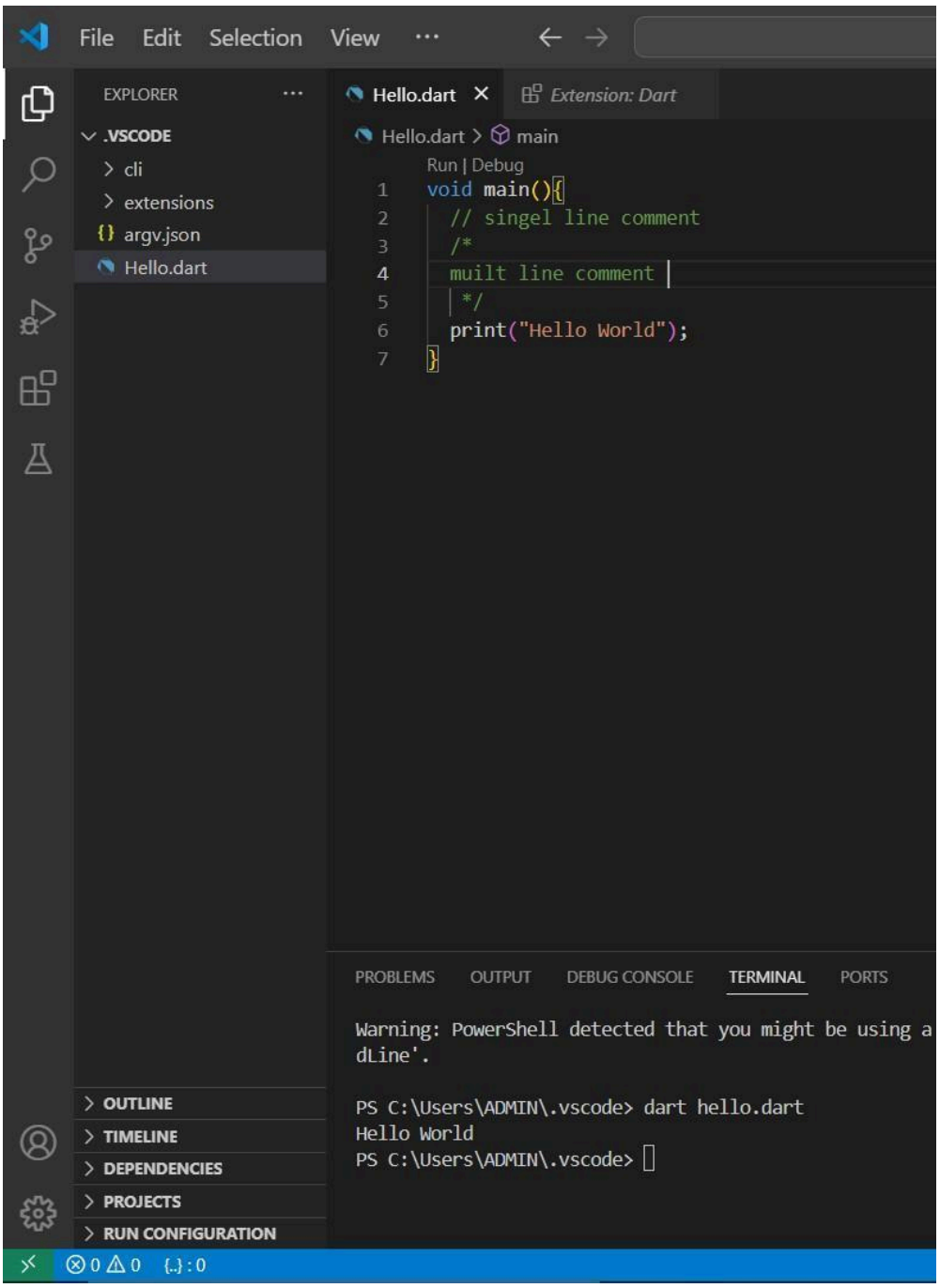
/* This is a block comment. It can contain multiple lines as comment. Another line in this block comment. */ }

Documentation Comments

- •These comments are similar to single line comments. But are specially treated by IDEs and dartdoc libraries. You can use these documentation comments for documenting your classes, methods, etc., in the dart libraries you develop.
- •Using some documentation preparation tools, you can generate documentation from code automatically

///Documentation Comments

///Some description about main() method. void main(){}



1.1 Dart Variable and Data type

Data Types

Numbers

Strings

Booleans

Lists

Maps

▪Number
It has two datatypes :Integer & Double

▪Strings
Strings represent a sequence of characters.

▪Boolean
The Boolean data type represents Boolean values true and false. Dart uses the bool keyword to represent a Boolean value.

▪List
- •A List is an ordered group of objects. The List data type in Dart is synonymous to the concept of an array in other programming languages.

▪Map
The Map data type represents a set of values as key-value pairs.

▪Dynamic Type
If the type of a variable is not explicitly specified, the variable's type is dynamic.

Variable
Method to Define Variable

Var

▪Datatype

▪Dynamic

▪Final

▪Const

var in Dart

Used to declare a variable whose value can change (mutable).

Dart infers the type automatically.

Example:

var name = 'Alice'; // String name = 'Bob'; // Allowed

- •A variable must be declared before it is used.
- •Dart uses the var keyword to declare the variable.

▪Example :-

var name = 'Darshan Gada';

Var Example

Void main(){

var a = 10;

print("A Value is $a");

}

Void main(){

var a = 10; var b = 20;

print("A Value is $a B Value is $b");

}

const in Dart

Used to declare a constant value known at compile time.

The value cannot be changed (immutable).

Example: const pi = 3.14;

// pi = 3.1415; // Error: cannot modify a const

Data Types in dart

Numbers (int, double)

Strings (String)

Booleans (bool)

((value1, value2))

Functions (Function)

Lists (List, also known as arrays)

Sets (Set)

Maps (Map)

Runes (Runes; often replaced by the characters API)

Symbols (Symbol)

The value null (Null)

Numbers

Dart numbers come in two flavors:

Integer values no larger than 64 bits, . On native platforms, values can be from -263 to 263 - 1. On the web, integer values are represented as JavaScript numbers (64-bit floating-point values with no fractional part) and can be from -253 to 253 - 1.

64-bit (double-precision) floating-point numbers, as specified by the IEEE 754 standard.

Both int and double are subtypes of . The num type includes basic operators such as +, -, /, and *, and is also where you'll find abs(),ceil(), and floor(), among other methods. (Bitwise operators, such as >>, are defined in the int class.) If num and its subtypes don't have what you're looking for, the library might.

Strings

A Dart string (String object) holds a sequence of UTF-16 (Unicode Transformation Format) code units. You can use either single or double quotes to create a string:

var s1 = 'Single quotes work well for string literals.'; var s2 = "Double quotes work just as well.";

var s3 = 'It\'s easy to escape the string delimiter.'; var s4 = "It's even easier to use the other delimiter.";

content_copy

You can put the value of an expression inside a string by using ${expression}. If the expression is an identifier, you can skip the {}. To get the string corresponding to an object, Dart calls the object's toString() method.

Example :-

var s = 'string interpolation';

assert(

'Dart has $s, which is very handy.' == 'Dart has string interpolation, '

'which is very handy.',

);

assert(

'That deserves all caps. ' '${s.toUpperCase()} is very handy!' ==

'That deserves all caps. '

'STRING INTERPOLATION is very handy!',

);

Note :- The == operator tests whether two objects are equivalent. Two strings are equivalent if they contain the same sequence of code units.

You can concatenate strings using adjacent string literals or the + operator:

var s1 =

'String ' 'concatenation'

" works even over line breaks."; assert(

s1 ==

'String concatenation works even over ' 'line breaks.',

);

var s2 = 'The + operator ' + 'works, as well.'; assert(s2 == 'The + operator works, as well.');

To create a multi-line string, use a triple quote with either single or double quotation marks:
var s1 = '''

You can create

multi-line strings like this one. ''';

var s2 = """This is also a multi-line string.""";

var s1 = '''

You can create

multi-line strings like this one. ''';

var s2 = """This is also a multi-line string.""";

You can create a "raw" string by prefixing it with r:

var s = r'In a raw string, not even \n gets special treatment.';

Booleans

To represent boolean values, Dart has a type named bool. Only two objects have type bool: the boolean literals true and false, which are both compile-time constants.

Dart's type safety means that you can't use code like if (nonbooleanValue) or assert (nonbooleanValue). Instead, explicitly check for values, like this:

// Check for an empty string. var fullName = ''; assert(fullName.isEmpty);

// Check for zero. var hitPoints = 0;

assert(hitPoints == 0);

// Check for null. var unicorn = null;

assert(unicorn == null);

// Check for NaN.

var iMeantToDoThis = 0 / 0; assert(iMeantToDoThis.isNaN);

Runes and grapheme clusters

In Dart, expose the Unicode code points of a string. You can use the to view or manipulate user-perceived characters, also known as Unicode (extended) grapheme clusters.

Unicode defines a unique numeric value for each letter, digit, and symbol used in all of the world's writing systems. Because a Dart string is a sequence of UTF-16 code units, expressing Unicode code points within a string requires special syntax. The usual way to express a Unicode code point is \uXXXX, where XXXX is a 4-digit hexadecimal value. For example, the heart character (♥) is \u2665. To specify more or less than 4 hex digits, place the value in curly

brackets. For example, the laughing emoji (😆) is \u{1f606}.

If you need to read or write individual Unicode characters, use the characters getter defined on String by the characters package. The returned object is the string as a sequence of grapheme clusters. Here's an example of using the characters API:

import 'package:characters/characters.dart';

```
void main() {

var hi = 'Hi '; print(hi);

print('The end of the string: ${hi.substring(hi.length - 1)}');
print('The last character: ${hi.characters.last}');
```

}

Symbols

An object represents an operator or identifier declared in a Dart program. You might never need to use symbols, but they're invaluable for APIs that refer to identifiers by name, because minification changes identifier names but not identifier symbols.

To get the symbol for an identifier, use a symbol literal, which is just # followed by the identifier:

#radix #bar

CH - 2 Operators

Arithmetic Operators

Relational Operators

Assignment Operators

Logical Operators

Type test Operators

Conditional Operators Arithmetic Operators

Example :-

Remainder %

PS C:\Users\ADMIN\.vscode> dart .\Hello.dart Division of 23 is 10 is 3

Increment/Decrement Operator

PS C:\Users\ADMIN\.vscode> dart .\Hello.dart N1 Value Before Increment is 10

N1 value After Increment is 11 N2 Value Before Decrement is 20 N2 Value after Increment is 19

Relational Operators

PS C:\Users\ADMIN\.vscode> dart .\Hello.dart N1 is greater than N2 = false

N1 is Less than N2 = true

N1 is greater than or euqal to N2 = false N1 is less than or Equal to N2 = true N1 is not Equal to N2 = true

N1 is Equal to N2 = false

Assignment Operators

Example code:

Type test Operators

Example:

Conditional Operators

- •The ternary operator is an operator that takes three arguments.

▪It can be represented with ? :

- •It is also called as conditional operator.
- •It is shortened way of writing an if-else statement.
- •Syntax:- expression-1 ? expression-2 : expression-3 condition execute if true execute if false
- •In the above symbol expression-1 is condition and expression-2 and expression-3 will be either value or variable or statement or any mathematical expression.
- •If condition will be true expression-2 will be execute otherwise expression-3 will be executed.

PS C:\Users\ADMIN\.vscode> dart Hello.dart N2 is Greater then N1

Logical Operator

true true False

2.1 Control Statements

Conditional Statements

If Statement

- •The if statement is used to perform operation on the basis of condition.
- •The single if statement is used to execute the code if condition is true.

If syntax if(condition){

True statement;

}

Example:

PS C:\Users\ADMIN\.vscode> dart Hello.dart Number is even

If Else Statement

- •The if-else statement is used to execute the code if condition is true or false.
- •It executes the if block if condition is true otherwise else block is executed.

Syntax :- if(condition) {

//Statement to be executed if condition is true

} else {

//Statement to be executed if condition is false

}

PS C:\Users\ADMIN\.vscode> dart Hello.dart Number is even

If – Else If Else Statement

- •The if else-if statement is used to execute one code from multiple conditions. Syntax :-

if(condition1) {

//Statement to be executed if condition1 is true

}else if(condition2) {

//Statement to be executed if condition2 is true

}else if(condition3) {

//Statement to be executed if condition3 is true

} ... else {

//Statement to be executed if all conditions are false

}

Example:

Switch

Switch Statement

- •The switch statement is used to execute the code from multiple conditions.
- •We can create menu driven program.
- •It is like if else-if statement.

Switch Syntax

- •Syntax :- switch(expression) {

case value1: //code to be executed; break;

case value2: //code to be executed; break; .....

default: //code to be executed if all cases are not matched

}

Example:-

Loops

While loop

Do…while loop

For loop

For…in loop

For…each loop

Types of Loops

▪Entry Control

▪Exit Control

While Loop

- •The while loop is used to repeat a part of the program several times.

If the number of iteration is not fixed, it is recommended to use while loop.

- •Syntax Of While Loop :- variable initialization; while(condition) {

//Code to be executed variable increment or decrement;

}

Example:

PS C:\Users\ADMIN\.vscode> dart Hello.dart 1

2

3

4

5

6

7

8

9

10

Do...While Loop

- •Do-while loop is used to repeat a part of the program several times.
- •If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.
- •The do-while loop is executed at least once because condition is checked after loop body.

- •Syntax Of Do...While Loop :- variable initialization;

do {

//Code to be executed variable increment or decrement;

}while(condition);

Example:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart 1

2

3

4

5

For Loop

- •The Java for loop is used to repeat a part of the program several times.
- •If the number of iteration is fixed, it is recommended to use for loop.

▪Syntax Of For Loop :-

for(initialization; condition; increment/decrement) {

//Code to be executed

}

Example:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart 0

1

2

3

4

5

6

7

8

9

10

For…in Loop

- •The for…in loop is used in array or collection.
- •It works on elements basis not index.

It returns element one by one Example:-

```
PS C:\Users\ADMIN\.vscode> dart Hello.dart 10
20
30
40
50
60
70
80
90
100
```

For each :- Syntax:-

```
collection.forEach((variable name){ statement
})
```

```
PS C:\Users\ADMIN\.vscode> dart Hello.dart 10
20
30
40
50
60
70
80
90
100
```

Ch - 3 Dynamic (input and output)

StdIn

Standard Functions

- •Dart offers us a standard library called 'io' that has many classes and methods for reading.
- •Using the import command, we can import the library into our program.
- •Import 'dart:io'; Standard Input
- •The .readLineSync() function in the Dart programming language allows you to take input from the user via the console.
- •To get input from the console, you will need to import the Dart:io library. Code

Output:

PS C:\Users\ADMIN\.vscode> dart Hello.dart Enter Your Name?

darshan

Hello, darshan!

Welcome to Dart!

Sum Demo

PS C:\Users\ADMIN\.vscode> dart Hello.dart Enter No1 :

5

Enter No2 : 21

Sum is 26

Standard Output

- •There are two possible ways to show output in the console in Dart.
- •By using the print statement,
- •By using stdout.write() method. Print Vs Stdout.Write
- •print() statement brings down the cursor to the next line(just like println() in java)
- •stdout.write() do not bring the cursor to the next line(just like simple print() of java); it remains in the same line.

3.1 List Collection

Lists

- •Dart represents arrays in the form of List objects.

- •A List is simply an ordered group of objects.
- •The dart:core library provides the List class that enables creation and manipulation of lists.

Fixed Length List

▪Growable List Fixed Length List

- •A fixed length list's length cannot change at runtime.

var list_name = new List(initial_size)

Example:-

OutPut :- PS C:\Users\ADMIN\.vscode> dart Hello.dart

[0, 0, 0, 0, 0]

[87, 1, 2, 3, 0]

Example 2 :-

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart [G, B]

[G, B, X, C, B]

List Properties

To check whether, and where, the element is in the list, use or . final indexA = growableList.indexOf('A'); // -1 (not in the list)

final firstIndexB = growableList.indexOf('B'); // 1

final lastIndexB = growableList.lastIndexOf('B'); // 4

To remove an element from the growable list, use , , , or .

growableList.remove('C'); growableList.removeLast();
print(growableList); // [G, B, X]

print(growableList); // [G, New, B, X]

```
growableList.replaceRange(0, 2, ['AB', 'A']); print(growableList); //
[AB, A, B, X]

growableList.fillRange(2, 4, 'F'); print(growableList); // [AB, A, F, F]
```

To sort the elements of the list, use .

```
growableList.sort((a, b) => a.compareTo(b)); print(growableList); //
[A, AB, F, F]
```

```
growableList.shuffle(); print(growableList); // e.g. [AB, F, A, F]
```

```
bool isVowel(String char) => char.length == 1 &&
"AEIOU".contains(char); final firstVowel =
growableList.firstWhere(isVowel, orElse: () => ''); // ''
```

3.2 Map Collection

Map

- •The Map object is a simple key/value pair. Keys and values in a map may be of any type. A Map is a dynamic collection.

▪Using Map Literals

- •Using a Map constructor Declaring a Map using Map Literals
- •To declare a map using map literals, you need to enclose the keyvalue pairs within a pair of curly brackets "{ }".

▪Syntax :-

- •var identifier = { key1:value1, key2:value2 [,…..,key_n:value_n] } Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart

{one: Name, Two: age}

Example: Adding Values to Map Literals at Runtime

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart

{one: Name, Two: age, three: Number} Map – Properties

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart (one, Two, three)

(Name, age, Number) 3

false true

Map - Functions

- •Map.addAll()

Syntax:-

Map.addAll(Map<k,v>)

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart

{one: Name, Two: age, three: Number, four: space, five: Free}

- •Map.clear()

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart

{}

- •Map.remove()

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart Name

{Two: age, three: Number}

- •Map.forEach() Map.forEach()

forEach enables iterating through the Map's entries.

▪Syntax :-

- •Map.forEach(f(K key, V value));

PS C:\Users\ADMIN\.vscode> dart Hello.dart one : Name

Two : age three : Number

3.3 Class OOP

Class & Interface

What is Class ?

Class is a programmer-defined data type, which includes local methods and local variables.

Class is a collection of objects. Object has properties and behavior.

> ***First we have to define a class, where the class name should be the same as the filename.***
>
> ***When class is created, we can create any number of objects in that class. The object is created with the help of the new keyword.***

How to create classes?

- •In order to create a class, we group the code that handles a certain topic into one place.

> ***We declare the class with the class keyword.***
>
> ***We write the name of the class and capitalize the first letter. ▪ Example : Car***
>
> ***If the class name contains more than one word, we capitalize each word. This is known as upper camel case***
>
> ***Example : Car_Class 4. We circle the class body within curly braces. Inside the curly braces,we put our code.***

Class Car {}

Example :- Class Car{

// Write your code here

}

Classes in Dart

- •A class in terms of OOP is a blueprint for creating objects. A class encapsulates data for the object.
- •Syntax :- class class_name {

<Fields>

<getter/setter>

<constructors>

<Function>

}

How to create objects from a class?

- •We can create several objects from the same class, with each object having its own set of properties.
- •In order to work with a class, we need to create an object from it.

In order to create an object, we use the new keyword.

▪Example :

- •bmw = new Car ();
- •mercedes = new Car (); Object in Dart
- •There are two ways to define object of a class. Using period operator (.) Using Cascade operator (..)

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart BMW

Example:- (Cascade Operator)

class Car {

```
void showName() {
        print("Show Method Called");
}
}
        void main() {
new Car()
..showName();
}
```

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart Show Method Called

Constructors

- •A constructor is a special function of the class that is responsible for initializing the variables of the class.
- •Dart defines a constructor with the same name as that of the class.
- •A constructor is a function and hence can be parameterized.

▪However, unlike a function, constructors cannot have a return type. Syntax:

Class_name(parameter_list) { //constructor body } Constructor Example

- •Constructor will automatically called when you call object of class. •Constructor name must be same as class name.

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart Car Constructor Called

```
        class Car {
```

```
//Car Constructor
constructor() {
        print("Car Constructor Called");
}
}
        void main() {
Car obj1 = new Car();
}
```

Output:-

```
PS C:\Users\ADMIN\.vscode> dart Hello.dart Car NameBMW
```

Constructor with Multiple Parameter

```
//Car Class
        class Car {
//Car Constructor
Car(String name, String model) {
        print("Car Constructor Called");
        print("Car Name: " + name);
        print("Car Model : " + model);
}
}
        void main() {
Car obj1 = new Car("BMW", "2022");
}
```

Named Constructors

- •Dart provides named constructors to enable a class define multiple constructors.
- •Syntax : Defining the constructor
- •Class_name.constructor_name(param_list)

//Car Class

```
class Car {
```

//Car Constructor

Car() {

```
print("Default Constroctor Called");
```

}

Car.MyConstroctor1(String name) {

```
print("MyConstroctor1 called value is " + name);
```

}

}

```
void main() {
```

Car obj1 = new Car();

Car obj2 = new Car.MyConstroctor1("Darsan");

}

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart This is my car class

```
This is my Same class another constructor The this Keyword
```

- •The this keyword refers to the current instance of the class.
- •Here, the parameter name and the name of the class's field are the same.

- •Hence to avoid ambiguity, the class's field is prefixed with the this keyword.

Example:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart Car Name: BMW

Class — Getters and Setters

- •Getters and Setters, also called as accessors and mutators, allow the program to initialize and retrieve the values of class fields respectively.
- •Getters or accessors are defined using the get keyword. Setters or mutators are defined using the set keyword.
- •A getter has no parameters and returns a value, and the setter has one parameter and does not return a value.

▪Syntax:

Defining a getter Return_type get identifier { }

- •Syntax: Defining a setter set identifier { }

print(obj.name);

}

The static Keyword

- •The static keyword can be applied to the data members of a class, i.e., fields and methods.
- •A static variable retains its values till the program finishes execution. •Static members are referenced by the class name.

Interfaces

- •Interfaces define a set of methods available on an object. Dart does not have a syntax for declaring interfaces. Class declarations are themselves interfaces in Dart.
- •Classes should use the implements keyword to be able to use an interface. •A class must redefine every function in the interface it wishes to implement.

▪Syntax:

Implementing an Interface

- •class identifier implements interface_name

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart This is your another class

```
Example Of Multiple Interfaces class Demo {
```

static void show() {

```
print("Static show from Demo");
```

}

}

```
class Demo2 { void show2() {

print("show2 from Demo2");
```

}

}

```
// Since Demo.show is static, we do NOT implement it in Demo3 class
Demo3 implements Demo2 {
```

@override void show2() {

```
print("Hello from Demo3's show2");
```

}

}

```
void main() {
```

// Call static method directly on class

Demo.show(); // Correct way to use a static method

// Use Demo3 which implements Demo2 Demo3 obj = Demo3();

obj.show2(); // Calls overridden method from Demo3

}

3.4 Functions

- •A function is a set of statements to perform a specific task.

You can divide up your code into separate functions.

- •A function can be called many times.
- •It provides code reusability.

Before using a function, it must be defined.

Function

Function with Return

Parameterized Function

Optional Parameters

Optional Parameters with Name

Optional Parameters with Default Values

Recursive Functions

Lambda Functions Syntax of defining functions

- •Syntax :- function_name() {

//statements

}

- •OR

void function_name() {

//statements

}

Calling a Function

- •A function must be called to execute it. This process is termed as function invocation.

▪Syntax :-

- •function_name()

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart I am Function

Returning Function

- •Functions may also return value along with the control, back to the caller. Such functions are called returning functions.

▪Syntax :-

return_type function_name(){

//statements return value;

}

Function Return Example

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart I am the function

Parameterized Function

- •Parameters are a mechanism to pass values to functions.
- •The parameter values are passed to the function during its invocation.

▪Syntax :-

Function_name(data_type param_1, data_type param_2) {

//statements

}

Parameterized Example

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart 30

Optional Positional Parameter

- •To specify optional positional parameters, use square [] brackets.

▪Syntax :-

void function_name(param1, [optional_param_1, optional_param_2]) { }

- •If an optional parameter is not passed a value, it is set to NULL. Optional Parameter Example

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart 10

[10, 20, 50]

null

- •The parameters' name must be specified while the value is being passed. Curly brace {} can be used to specify optional named parameters.
- •Syntax - Declaring the function

void function_name(a, {optional_param1, optional_param2}) { }

- •Syntax - Calling the function

function_name(optional_param:value,…);

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart 10

null null 10

20

null 10

20

30

Optional Parameters with Default Values

- •Function parameters can also be assigned values by default. However, such parameters can also be explicitly passed values.
- •Syntax :- function_name(param1,{param2= default_value}) {

//.....

}

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart 10

Default 10

20

Recursive Functions

- •Recursion is a technique for iterating over an operation by having a function call to itself repeatedly until it arrives at a result.
- •Recursion is best applied when you need to call the same function repeatedly with different parameters from within a loop.

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart 120

Lambda Functions

- •Lambda functions are a concise mechanism to represent functions. These functions are also called Arrow functions.
- •Syntax :- [return_type]function_name(parameters)=>expression;

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart Hello World

Darshan 10

40

Chapter - 4 Inheritance

Inheritance

§ Inheritance is a mechanism in which one object acquires all the properties and behaviors of parent object.

§ A class inherits from another class using the 'extends' keyword.

§ Child classes inherit all properties and methods except constructors from the parent class.

§ Syntax :-

§ class child_class_name extends parent_class_name

Example

```
class Demo{
}

class Demo1 extends Demo
{
//methods and fields goes here
}
```

Here Demo is Super or Parent Class and Demo1 is child class

Types of inheritance

§Single inheritance

§Multilevel inheritance

§Hierarchical inheritance

§Multiple inheritance

Single Level Inheritance

> ***§When one class inherits another class, it is known as single level inheritance.***

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart Method 1 Called

Method 2 Called

Multi Level Inheritance

> ***§When one class inherits another class which is further inherited by another class, it is known as multi level inheritance.***

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart Hello.dart Method 1 Called

Method 2 Called

Method 3 Called

Multiple Inheritance

> ***§A class can inherit from multiple classes. Dart doesn't support multiple inheritance.***
>
> ***Dart doesn't directly support multiple implementation inheritance (where a class inherits from multiple parent classes), but it does support multiple interface inheritance. To achieve similar results, you can utilize mixins. Mixins allow you to include the functionality of one class into another, effectively mimicking the behavior of multiple inheritance.***

Here's how it works:

Define Mixins: Create classes that serve as mixins. These classes should contain the code (methods, variables) that you want to incorporate into other classes.

Include Mixins: Use the with keyword to include one or more mixins into a class.

> ***Inheritance (optional): You can also extend a parent class using the extends keyword, combining inheritance and mixins.***

Example :-

// Abstract Base class

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Log: Doing something

Error: An error occurred

Hierarchical Inheritance

> ***§If more than one class can inherit members from one class is known as Hierarchical Inheritance.***

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Method 1 Called

Method 3 Called

Method 1 Called

Method 2 Called

The super Keyword

§The super keyword is used to refer to the immediate parent of a class.

> ***§The keyword can be used to refer to the super class version of a variable, property, or method.***

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Method 1 Called

Method 2 Called Super Variable

Class Inheritance and Method Overriding Method Overriding is a mechanism by which the child class redefines a method in its parent class

---

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Method 1 Called from class Demo2

Method 2 Called

4.1 String

String Functions

- •The String data type represents a sequence of characters. Syntax :-

String variable_name = 'value' OR

String variable_name = ''value'' OR

String variable_name = '''line1 line2''' OR

String variable_name= '''''line1 line2''''' String Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart First Methode

Second Methode Third Methode Fourth Methode

isEmpty & isNotEmpty

- •Returns true if this string is empty.

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart false

true Length

- •Returns the length of the string including space, tab and newline characters. Exampel:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 13

toUpperCase() toLowerCase()

toUpperCase()

Converts all characters in this string to upper case.

toLowerCase()

Converts all characters in this string to lower case.

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart first methode

FIRST METHODE

trim()

- •Returns the string without any leading and trailing whitespace. However, this method doesn't discard spaces between two strings.

Example:-

```
void main() {

var myString1 = '    First Methode  '; print(myString1.trim());
print(myString1.trimLeft()); print(myString1.trimRight());
```

}

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart First Methode

First Methode

First Methode indexOf(), lastIndexOf()

- •Returns the position of the first and last matches of the given pattern: Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 12

6

split()

- •Splits the string at the matching pattern, returning a list of substrings: Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart [First M, thod,]

[F, i, r, s, t, , M, e, t, h, o, d, e] [First Methode]

replaceAll() , replaceFirst() replaceRange()

▪Replaces all substrings that match the specified pattern with the replacement string:

- •ReplaceFirst Will Replace First Character Only.
- •ReplaceRange will Replace range Word with new One. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart First Methede

First Methede Welcome Methode endsWith()

▪Checks whether the string ends with the specified characters:

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart false

false true

startsWith()

- •Checks whether the string Starts with the specified characters: Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart true

false False

indexOf()

- •indexOf() will return index of character Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 0

3

6

substring()

- •Get Substring from String. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart First Methode

First Methode

Contains()

- •Contains will check String/Character is present or not.

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart true

true true

Join/Concatenate Strings Example:-

```
void main(){

var a = 'Hello';

var b = 'World';

String s1 ='$a$b';

String s2 = '$a' '$b';

String s3 = a+b;

String s4 = a*3;

print(s1);

print(s2);

print(s3);

print(s4);

}
```

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart HelloWorld

HelloWorld HelloWorld HelloHelloHello

compareTo()

Compares this object to another.

Returns an integer representing the relationship between two strings.

toString()

- •Returns a string representation of this object.

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 101

true

codeUnitAt()

- •Returns the 16-bit UTF-16 code unit at the given index.

https://unicode-table.com/en/#0048

Syntax :- ▪ String.codeUnitAt(int index) Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart [72, 101, 108, 108, 111]

108

4.2 Number

Number Functions

abs()

Example:-

- •Returns the absolute value of the number.

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 99

ceil()

- •Returns the greatest integer not greater than the current number. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 100

floor()

- •Returns the least integer no smaller than the number. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 99

remainder()

- •Returns the truncated remainder after dividing the two numbers. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 0

Truncate

- •Returns an integer after discarding any fractional digits.

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 20

toString()

Example:-

- •Returns the string equivalent representation of the number.

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 20.65

true

toInt()

- •Returns the Integer equivalent representation of the number. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 20

true

toDouble()

- •Returns the Double equivalent representation of the number. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 20.65

true

compareTo()

compareTo()

- •Compares this to other number.

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 0

Number is equal

Other Properties Parse()

- •The parse() function converts the numeric string to the number. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Sum 100

isFinite isInfinite

- •isFinite If the given number is finite, then it returns true.
- •isInfinite If the number infinite it returns true. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart false

true

isNegative

Example:-

- •If the number is negative then it returns true.

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart true

true

4.3 Library

Libraries in Dart

- •A library in a programming language represents a collection of routines (set of programming instructions).
- •A Dart library comprises of a set of classes, constants, functions, typedefs, properties, and exceptions.

Syntax:- Import ‘uri;

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Square Root of 25 is: 5.0

Custom Library

- •Step 1: Declaring a Library

▪Syntax :-

library library_name

- •Step 2: Associating a Library

Within the same directory

import 'library_name'

From a different directory

import 'dir/library_name' Example:-

main.dart file

my_lib.dart

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart inside add method of my_library

50

Library Prefix

- •If you import two libraries with conflicting identifiers, then you can specify a prefix for one or both libraries.
- •Use the 'as' keyword for specifying the prefix.

▪Syntax :-

- •import 'library_uri' as prefix

Create New Library

- •First, let us define a library: demo_library.dart Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart inside add method of my_library

50

This is your another demo

Chapter - 5 Async Await Future

Async Await Future When to Use ?

- •file i/o (Downloading File)
- •some sort of computation
- •API call to a RESTful service Asynchronous
- •Any functions you want to run asynchronously, needs to have the async modifier added to it. This modifier comes right after the function signature,

Syntax:-

void hello() async {

print('something exciting is going to happen here...');

}

Await

- •The await part basically says — go ahead and run this function asynchronously, and when it is done, continue on to the next line of code.

Example:-

void main() async { await hello(); print('all done');

}

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Task Start

Print Msg Called Task Complete

Future

- •Futures are a way to wrap something that will be resolved later on, so that we don't need to wait for it to follow along with our program.

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Task Start

Print Msg Called Task Complete

Without Await

Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart Task Start

Task Complete Print Msg Called

5.1 Null Safety

- •Null safety simply means that a variable cannot have a null value unless you initialized it with a null value.
- •Null safety makes types in your Dart code non-nullable naturally.
- •It implies that factors can't contain null except if you say they can. Dart's null safety is sound.

This implies that Dart is 100% certain that the records list, and the components in it, can't be null.

Declaring Non-Nullable Variables

▪When using non-nullable variables, we must follow one important rule:

- •Non-nullable variables must always be initialized with non-null values. Example:-

Declaring Nullable Variables (?)

- •To specify if the variable can be null, we use the nullable type ? operator. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart null

null null

Assertion Operator(!)

- •To specify the nullable variable as non-null use the null assertion ! operator.

int? maybeValue = 42;

int value = maybeValue!; // valid, value is non-nullable

5.2 Collection

Collections & Generics

- •Dart doesn't support arrays.
- •Dart collections can be used to replicate data structures like an array.
- •The dart:core library and other classes enable Collection support in Dart scripts.
- •Dart collections can be classified as follow :-

▪List

▪Set

▪Maps

▪Queue

List

- •A List is simply an ordered group of objects.

Set

- •Set represents a collection of objects in which each object can occur only once.

▪Syntax :-

- •Identifier = new Set()
- •OR
- •Identifier = new Set.from(Iterable)

Example:-

Output:

PS C:\Users\ADMIN\.vscode> dart run main.dart 10

20

30

Darshan Gada 50

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart 10

20

30

Maps

- •The Map object is a simple key/value pair. Keys and values in a map may be of any type.
- •A Map is a dynamic collection. In other words, Maps can grow and shrink at runtime.

Generics

- •The type-safe implementations of List, Map, Set and Queue is known as Generics.

▪Syntax :-

Collection_name identifier= new Collection_name

Example:-

Output:- a

b c d

JSON

dart:convert

- •Dart language has a built-in support for parsing json data.

You can easily convert json data into String using dart:convert library and map it with key: value parse, where keys are string and value are of dynamic objects.

> ***import 'dart:convert';***
>
> ***You need to import dart:convert to convert json data.***

5.3 JSON

JSON Encode

▪Json Encode

- •Object convert into JSON

▪Json Decode

JSON Data Convert into Dart Object Dart/Flutter convert List to JSON string Example:-

Output:-

["A","B","C","D","E","F","G","H"]

Dart/Flutter convert simple Object to JSON string

- •dart:convert library’s built-in jsonEncode() function to convert the User object to JSON string: Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart

{"name":"Akash","age":11}

JSON Decode

Direct Json Parsing and How to use the data

- •JSON Data Print var jsonData = '{ "name" : "Akash", "website" : "akashsir.com" }'; //simple json data var parsedJson = json.decode(jsonData); // need to import convert library print('${parsedJson.runtimeType} : $parsedJson'); //Display full json Parse print(parsedJson['name']); //fetch value using key 'name' print(parsedJson['website']); // fetch value using key 'website

JSON Parse in Dart

Json.decode() will convert Json Object into Dart Object .

We can Print Value by using Key. Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart

_InternalLinkedHashMap<String, dynamic>:{name: Darshan, website: darshan}

Darshan darshan

Dart Map

- •In Dart or any other programming language, A Map Object is a keyvalue pair to store string or any dynamic data.
- •Here, In dart map, A key & value can be of any datatype, It means that we can store any data type because a map in dart language is a dynamic collection of data/information.

▪Syntax

var identifier = { ‘key1’:’value1’ , ‘key2’:’value2’ , ‘key3’:’value3’, };

JSON Parser using Object Class

Instead of using json Parser directly, it's better to pass the json data to a class that handles your data for you. This can be done using a constructor in flutter dart

Class

parsed Json data using json.decode and then Created a class object names userData and passed parsed json data to class constructor.

class UserData { String name; String website;

UserData(Map<String, dynamic> data) { name = data['name'];

website = data['website'];

}

}

var jsonData = '{ "name" : "Akash", "website" : "akahsir.com" }'; //simple json data var parsedJson = json.decode(jsonData); // need to import convert library

UserData userData = UserData(parsedJson); // creating an object and passed parsed JSON data print("${userData.name} Keep learning from ${userData.website}");

We can read data

▪Dot Notaion

userData.name

▪Array Notation

userData['name']

JSON Decode with Class in Dart Example:-

Output:-

PS C:\Users\ADMIN\.vscode> dart run main.dart

Name is :Darshan Website is :darshan123.com Mobile Number is :9512714369

Chapter - 6 Introduction to Flutter Framework

Flutter UI Kit

App Development

What is a Native App?

- •Native apps are built specifically for a particular operating system leveraging platform-specific programming languages.
- •Native app development is the process to develop apps or software that need to be operated on specific devices and mobile app platforms such as Android and iOS.

▪Programming:

- •Android (Android Studio): Java ,Kotlin,

iOS (XCode) : Swift, Objective-C Benefits of Native Apps

▪High Speed

▪Offline Functionality

▪Interactive

- •Minimized Scope of Bugs

▪Enhanced Security

What is a Hybrid App?

- •Hybrid app development is the combination of native and web solutions where developers need to embed the code written with the languages such as CSS, HTML, and JavaScript into a native app with the help of plugins including Ionic's Capacitor, Apache Cordova, and so on which enables to get the access of native functionalities.

▪Tools:

▪Apache Cordova

Ionic (Angular)

Benefits of Hybrid Apps

- •Rapid Time to Market

▪Easy Maintenance

▪Minimized Cost Backed with Ease of Development What is a Cross Platform App?

- •Cross-platform frameworks operate on the agenda to develop shareable and reusable code for building apps for different OS.
- •Writing code once and reusing the same on multiple platforms helps in minimizing the development costs and efforts.

▪Tools:

- •React Native (NodeJS,ReactJS)

▪Xamarin (C#)

Flutter (Dart)

6.1 Introduction

What is Flutter?

- •Flutter is an app SDK for building high-performance, high-fidelity apps for iOS, Android, and web from a single codebase.
- •Single Code base Means We have to write code and app will be run on Android , iOS Devices.
- •The goal is to enable developers to deliver high-performance apps that feel natural on different platforms.
- •We embrace differences in scrolling behaviours, typography, icons, and more.

Why use Flutter?

- •Develop for iOS and Android from a single codebase
- •Do more with less code, even on a single OS, with a modern, expressive language and a declarative approach
- •Flutter apps look and feel great.
- •Flutter is the only mobile SDK that provides reactive views without requiring a JavaScript bridge.

Flutter Current Version

3.29.1

Advantages

- •Faster User interface Coding

▪Hot Reload

- •One codebase for 2 platforms
- •Requires less testing
- •Best-suited for MVP: MVP means Minimum Viable Product
- •High performance
- •Quick to learn

Limitations

Flutter's limitations include larger app sizes compared to native apps, a potentially steeper learning curve for developers unfamiliar with Dart, and a slightly less extensive ecosystem of third-party libraries and plugins compared to more mature frameworks. Additionally, while Flutter has made significant progress, it may still experience performance bottlenecks in certain complex or graphics-intensive scenarios.

Hot reload

- •One of the most interesting features of the Flutter framework is its hot reload. It provides reloading the code on the running app without losing the state, massively decreasing development time.
- •Hot reloaded in an instant Make changes to one of your project's Dart files. A hot restart can be used to make a list of modifications.

If you're working in an IDE/editor that supports Flutter's IDE tools, Save All (cmd-s/ctrl-s), or click the Hot Reload button on the IDE's toolbar.

Hot restart

- •Hot restart is slightly distinct from hot reload. The hot restarts get the application fully compiled and all previous states will be set to their defaults. The app widget tree will be completely rebuilt. The hot restart needs more time than Hot reload.
- •Note: Only Flutter apps in debug mode can be Hot reloaded/Hot restarted.

Everything is a Widget

- •In the flutter app in the UI part, everything is a widget. Even the app itself is a widget!.
- •If you are familiar with Android and IOS development, widgets are like views in android and UIViews in IOS.
- •Flutter has a consistent, unified object model that is a widget.

Flutter Layout

Prerequisites, first.

- •OOP concepts
- •Dart language

https://dartpad.dev/

Hardware and Software Tools Required

Operating Systems:

Windows 7 SP1 or later (64-bit), x86-64 based.

Disk Space:

1.64 GB (does not include disk space for IDE/tools).

Tools:

Flutter depends on these tools being available in your environment.

Windows PowerShell 5.0 or newer (this is pre-installed with Windows 10) ▪ Git for Windows 2.x, with the Use Git from the Windows Command Prompt option.

RAM : 8 GB HDD : 160 GB

SSD (Best)

6.2 Flutter Install

Installation ☺ Hardware Requirement

▪Hardware:

- •8GB RAM
- •128 GB /256 GB SSD

Software

Android Studio

Download SDK

Download AVD

Flutter Plugin

Dart Plugin

- •Download Flutter SDK

▪Git Bash / Git

- •VS Code

Flutter Plugin

Dart Plugin

Material Icon Theme

System Requirements 8 GB Ram

1.6 GB minimum Free space in ssd or hdd

Software Requirements

Download Flutter SDK

Environment variablesCheck Flutter environment

Download and Setup Android

Download and Extract

▪Download Flutter

- •Go to extracted folder “flutter->src”
- •C:\flutter\src

▪Extract Flutter Files

Copy Path

F:\Flutter_Drive\FLUTTER_SETUP\flutter_windows_3.27.1-stable\flutter\bin

Set Environment Variable

Click on Environment Variables

Click on new add path of your flutter > src folder

Check Flutter Command

Flutter Default Commands

Flutter Version

Flutter Upgrade

Flutter Doctor

This command checks your environment and displays a report of the status of your Flutter installation.

before your setup is completed

After your setup is completed

Download android studio and java version from below link

Enter your project name instead of untitled and hit create

Setup in visual studio code

Click of extension panel and search for flutter and dart extension and install it

Now open the location of your project

It will be opened like this

Pubspec.ymal file

Quick Steps

- •flutter create myflutterproject
- •flutter run
- •flutter emulators
- •flutter emulators --launch
- •flutter emulators --create [--name xyz]
- •flutter emulators --launch flutter_emulator
- •flutter run

Commands

- •Flutter create
- •Flutter emulators
- •Flutter emulators --launch
- •Flutter run
- •Flutter run –d devicename
- •flutter doctor --android-licenses
- •Flutter upgrade

Other Command

- •flutter upgrade – upgrade Flutter SDK to latest version

- •flutter downgrade – downgrade Flutter SDK to lower version
- •flutter logs – Shows logs generated by flutter apps in emulators and devices.
- •flutter screenshot – Takes screenshot from emulator or device
- •flutter emulators – shows available emulators.
- •flutter devices – shows connected emulators and devices
- •flutter clean – reduces project size by deleting build and .dart_tool Directories.

6.3 Create New Project

Create Project in VS Code

- •Download VS Code
- •Download Flutter and Dart Extension in VS Code

Open VS Code

Download flutter Extension

Download dart Extension

Download Awesome Flutter Snippet

Create project

Ctr+shift+p shortcut for opening panel

Select Applcation

Select Application Option From Menu

Select Location to create new project

Give Application Name

Specify Application Name and hit enter

Wait for Few Minutes Open Main.dart file

Select Device to Run project (Pixel)

Select Device to Run Project or Create New Android Emulator

Open AVD

Run using command

Click on Debug Option to Run App

App Will be Ready

App Restart Option

Directory Structure

▪android :

- •where Android-related files are stored.

If you've done any sort of cross-platform mobile app development before, this, along with the ios folder should be pretty familiar.

▪ios :

- •where iOS-related files are stored.

Test :

this is where you put the unit testing code for the app.

▪lib :

this is where you'll be working on most of the time.

By default, it contains a main.dart file, this is the entry point file of the Flutter app.

▪pubspec.yaml :

this file defines the version and build number of your app.

It's also where you define your dependencies.

If you're coming from a web development background, this file has the same job

description as the package.json file so you can define the external packages (from the

Dart packages website) you want to use in here. Folder Direcotry

Import Files form folder

Lib

Main.dart

Flutter Application Code

Pubspec.yaml

Dependancy Management

Pubspec.yaml

The pubspec.yaml file is the project's configuration file you'll use a lot when working with your Flutter project. This file contains:

general project settings like name, description and version of the project

project dependencies

assets (e.g. images) which should be available in your Flutter application

Pubspec.yaml

Main.dart

- •Inside the lib folder you'll find the Dart files which contain the code of your Flutter application.
- •So this is the folder where you'd spend most of the time when implementing Flutter applications.
- •By default this folder is containing the file main.dart: Main.dart

6.4 Life Cycle

Flutter App Lifecycle

Life Cycle

- •inactive

The application is in an inactive state and is not receiving user input. This event only works on iOS, as there is no equivalent event to map to on Android

- •paused

The application is not currently visible to the user, not responding to user input, and running in the background. This is equivalent to onPause() in Android

Life Cycle

- •resumed

The application is visible and responding to user input. This is equivalent to onPostResume() in Android

- •suspending

The application is suspended momentarily. This is equivalent to onStop in Android; it is not triggered on iOS as there is no equivalent event to map to on iOS

Example @override

void initState() { WidgetsBinding.instance.addObserver(this); super.initState();

}

@override

```
void dispose() { WidgetsBinding.instance.removeObserver(this)
;
super.dispose();
}
```

Chapter - 7 Mirrors in Flutter

7.1 Stateless and Stateful Widget

Widget

- •Widget is a basic building block on flutter code.
- •Widgets are just pieces of your user interface. Text is a widget. Buttons are widgets. Check boxes are widgets. Images are widgets. And the list goes on. In fact, everything in the UI is a widget. Even the app itself is a widget!

'Everything is a widget' Widget trees

Types of Widget

StatelessWidget StatelessWidget is immutable once it is built.

StatefulWidget StatefulWidget is changed dynamically and can be redrawn within life cycle

What is a State?

- •The state is information that can read simultaneously when the widget is built and might change during runtime, in short, we can say that the State defines the current properties of the Widget.

Stateless Widget

- •Stateless widgets are widgets that don't store any state. That is, they don't store values that might change.
- •A stateless widget cannot change its state during the runtime of a Flutter application
- •For example, an Icon is stateless; you set the icon image when you create it and then it doesn't change any more.

Steps to implement Stateless Widget?

Create a class that extends 'StatelessWidget'.

Create a 'build()' method for widgets that never change their values during runtime.

'build()' method returns widget.

Statefull Widget

- •That means it can keep track of changes and update the UI based on those changes.
- •A stateful widget is useful for something like a checkbox.

▪When a user clicks it, the check state is updated Steps to implement Stateful Widget?

> ***Create a class that extends 'StatefulWidget', that returns state in 'createState()'***
>
> ***Create a 'State' class for widgets that may change their values during runtime.***

Within the 'State' class, implement the 'build()' method.

Call 'setState()' function. 'setState()' function actually redraws widgets.

Steps

Stateless VS Statefull

Widgets

Main vs RunApp

- •main()

We know that any programming language has the main entry point. This means that this is where our program first gets programmatically executed. So in dart we also have this, main () function.

- •runApp()

runApp() method is used to initialize and run the app. It takes in a Widget as its argument, which is typically the root of the app's widget hierarchy.

The runApp() method is typically called in the main() function of a Flutter app, which is the starting point of the app's execution.

The main() function is the entry point of the app and it is the first method that is executed when the app starts.

- •Open the main.dart file. It's in the lib folder in your project outline. •Delete all the text in this file and replace it with

> ***void main() {}***

- •If you hot reload your app now it should be a blank screen.
- •The main() function is the starting point for every Flutter app.

Simple Material App

import 'package:flutter/material.dart'; void main() {

runApp(const MaterialApp(home: Text("Welcome to Flutter"),

));

}

Container

import 'package:flutter/material.dart'; void main() {

runApp(Container(color: Colors.amber,

));

}

Stateful Widget

7.2 State

We can define variable and Stores values inside Statefull widget.

State Management

We can define and print Variable in Stateless widget.

To Update the State Value we need to use Stateful widget.

Variable Define and Use

Sum of 2 Number

Counter Example

setState(() {

no1 = no1 + 1; name = "Aarav";

});

Show Name on Button Click

Change Multiple Value

We can change Multiple Value on SetStat

Increment using Method

Toggle Button

Toggle Code

Text(is_visible ? "True" : "Flase"), ElevatedButton(onpressed(() {

setState(() {

is_visible = !is_visible;

});

}),

child:

Text(is_visible ? "Click to Flase" : "Click to

True")),

StateFul

Stateless and Stateful Widget

Chapter 8 Scaffold

Material App Widget

The “MaterialApp” widget can have many properties. We’ll only use the “home” property to define the “Scaffold” which will basically cover the entire main screen of the app. If you want to

know more about Scaffold then check out the documentation .

This Scaffold contains a number of useful properties to make your app UI more beautiful. The first one we will be using is “appBar” property to define the “AppBar” widget.

If you run the app now then you will see an empty app bar with blue background [which is the default color] and body with white background.

8.1 What is Scaffold?

A Scaffold Widget provides a framework which implements the basic material design visual layout structure of the flutter app.

It provides APIs for showing drawers, snack bars and bottom sheets.

appBar

An appBar is to display at the top of the scaffold it’s one of the primary content of Scaffold without which a scaffold widget is incomplete.

It defines what has to be displayed at the top of the screen. appBar uses the AppBar widget properties through Scaffold.appBar.

This AppBar widget itself has various properties like

title,

elevation,

brightness etc to name a few.

AppBar

APP BAR

```
import 'package:flutter/material.dart';

void main() {
runApp(const MyApp());
}

class MyApp extends StatelessWidget {
const MyApp({Key? key}) : super(key: key);

@override
Widget build(BuildContext context) {
return MaterialApp(
title: 'Flutter Demo',
theme: ThemeData(
primarySwatch: Colors.blue,
visualDensity: VisualDensity.adaptivePlatformDensity,
```

),

home: Scaffold(

appBar: AppBar(

title: const Text('Flutter Demo'),

),

body: const Center(

child: Text('Hello World'),

),

),

);

}

}

Scaffold Widget

8.2 Simple Text Widget

Remove Debug Banner Tag

While you're developing a Flutter application and running the application in debug mode, usually you will see a debug banner at the top right of the application, as shown in the picture below.

MaterialApp(debugShowCheckedModeBanner: false,

// other arguments

TextStyle Properties

style: TextStyle(

fontSize: 50,

color: Colors.black, fontStyle: FontStyle.italic, letterSpacing: 10,

fontWeight: FontWeight.bold, decoration: TextDecoration.underline, decorationColor: Colors.red,

decorationStyle: TextDecorationStyle.dotted, wordSpacing: 10, backgroundColor: Colors.yellow,

)))),

Change Theme

AppBar

import 'package:flutter/material.dart'; void main() {

runApp(const MyApp());

}

class MyApp extends StatelessWidget { const MyApp({super.key}); @override

Widget build(BuildContext context) { return MaterialApp(title:

"MyApp", home:

Scaffold(

appBar: AppBar(

title: const Text("MyFirstApp"),

)),

);

}

}

Text Widget

Background Color Change

Column

Rows

TextText

To display text in flutter applications we have to use a text widget

Text Widget Properties

style

textAlign

textDirection

softWrap

overflow

textScaleFactor

maxlines

style

> ***The text style property of the Text widget is used to apply different styles to the text. The style parameter uses the TextStyle class to apply styles.***

color

fontSize

fontWeight

fontStyle

letterSpacing

wordSpacing

height

foreground

background

shadows

decoration

decorationColor

decorationStyle

fontFamily

FontSize,Color,FontWeight

Color

fontSize

By default the text of the text widget is small. We will use fontSize when we want to increase or decrease the size of the text.

fontWeight

The fontWeight parameter is used to define whether the text should be normal, bold, or values ranging from w100 to w900.

body: Center(child: Column(

children: canst i

Text(

'Hello Flutter', style: TextStyle(

fontSize: 30,

color: DColors.deepOrange,

eight: FontWeight.bold), // TextStyle

Android Emulator - flutter_emulator:5554

), // Text

·1- ,

), II Column

), II Center

Letter Spacing,Word Spacing,FontStyle

letterSpacing

The letterSpacing is used to define the amount of space between the letters.

wordSpacing

The wordSpacing is used to define the amount of space between the words. It will be useful if the words are too close.

fontStyle

The fontStyle is used to define the style for the font which can be normal or italic.

Letter Spacing,Word Spacing,FontStyle

shadow

The Shadow constructor has three arguments color, offset, and blurRadius.We can add multiple shadows using shadows[Shadow1, Shadow2].

Color is color of the shadow.

Offset is direction from the origin of the text widget.

BlurRadius is the amount of space to be blurred.

Shadow

Decoration

This decoration parameter is used to decorate the text using TextDecoration class. It has three constants lineThrough, overline, and underline.

TextDecoration.lineThrough

TextDecoration.overline

TextDecoration.underline

decorationColor

Underline,Overline and LineThrough

decorationStyle

The decorationStyleparameter uses TextDecorationStyle class to apply a style to the text

double,

dashed,

dotted,

solid,

wavy

textAlign

The textAlign parameter aligns the text position in a parent widget. The constants of TextAlign are start, end, left, right, center, and justify.

TextAlign.start

TextAlign.end

TextAlign.center

TextStyle Properties

style: TextStyle(

fontSize: 50,

color: Colors.black, fontStyle: FontStyle.italic, letterSpacing: 10,

fontWeight: FontWeight.bold, decoration: TextDecoration.underline, decorationColor: Colors.red,

decorationStyle: TextDecorationStyle.dotted, wordSpacing: 10, backgroundColor: Colors.yellow,

))),

Text Widget Property Demo

Font Properties

Text(

'Akash',

style: TextStyle(decoration:TextDecoration.lineThrough),

)

Text('2',style:TextStyle(fontFeatures:[FontFeature.supersripts()]),),

Text('5', style: TextStyle(fontFeatures: [FontFeature.superscripts()]),),

RichText

In Flutter, you can display a paragraph text that has multiple different styles by using a RichText widget and a tree of TextSpan widgets in combination. The text may be on a single line or multiple lines based on the layout constraints.

RichText(

text: const TextSpan(children: [

TextSpan(

text: 'Hello',

style: TextStyle(color: Colors.red, fontSize:

30)),

TextSpan(

text: ' World',

style: TextStyle(fontSize: 30)), // this is

invisible

]),

),

Change Theme

Dark Theme

TextField

KeyBoard Type

TextInputType.text (Normal complete keyboard)

TextInputType.number (A numerical keyboard)

TextInputType.emailAddress (Normal keyboard with an “@”)

TextInputType.datetime (Numerical keyboard with a “/” and “:”)

TextInputType.numberWithOptions (Numerical keyboard with options to enabled signed and decimal mode)

TextInputType.multiline (Optimises for multi-line information)

Basic Layout

Icon

child: Icon(

Icons.add_box,

color:Colors.red,

size: 50.0,

),

Basic layout widgets (multiple children)

The widgets above only took one child. When creating a layout, though, it is often necessary to arrange multiple widgets together. We will see how to do that using rows, columns, and stacks.

Rows are easy. Just pass in a list of widgets to Row's children parameter.

Rows and columns

Alignment

StateFul Widget

'*import 'package:flutter/material.dart';*

void main(){ runApp(new MyApp());

StateLess and StateFull Widget

}

//StateLess

class MyApp extends StatelessWidget { @override

Widget build(BuildContext context) { return new MaterialApp(

title: "My App",

home: new MyScreen(),

);

}

}

//StateFul

class MyScreen extends StatefulWidget { @override

_MyScreenState createState() => _MyScreenState();

}

class _MyScreenState extends State<MyScreen> { @override Widget build(BuildContext context) { return new

Scaffold(

appBar: new AppBar(

title: new Text("My First App"),

),

body: new Center(child: new Text("Welcome to App"),),

);

}

}

Tab

import 'package:flutter/material.dart'; void main() => runApp(MyApp()); class MyApp extends StatelessWidget {

@override

Widget build(BuildContext context)

{ return MaterialApp(

title: 'Flutter Demo', theme: ThemeData(

primarySwatch: Colors.blue,

),

home: MyHomePage(),

);

class _MyHomePageState extends State<MyHomePage> { @override Widget build(BuildContext context) { return

DefaultTabController(length: 2, child: Scaffold(appBar:

AppBar(bottom:

TabBar(

tabs: [

Tab(icon: Icon(Icons.sms), text: "SMS",), Tab(icon: Icon(Icons.phone_iphone), text: "Call"),

}

}],

),

class MyHomePage extends StatefulWidget { @override

_MyHomePageState createState() => _MyHomePageState();

title: Text('Tab Bar Demo'),

),

body: TabBarView(

Center(child: Text("SMS Me on 123")),

Login

child: ListView(

children: <Widget>[

Container(

alignment: Alignment.center, padding: Edgeinsets.all(10), child: Text(

'Sign in',

style: TextStyle(fontSize: 20),

)), II Text, Container

Container(

padding: Edgeinsets.all(10), child: TextField(

controller: nameController, decoration: InputDecoration(

border: OutlineinputBorder(),

labelText: 'User Name',

controller: passwordController, decoration: InputDecoration(

border: OutlineinputBorder(),

labelText: 1Password1,

), II InputDecaration

), II TextField

), II Container

Container(

hei,ght: 5©,

padding: Edgeinsets.fromLTRB(l©, 10, 10, ©),

child: RaisedButton(t,extC,olor: Colors.white, color: Colors.blue, child: Text(1Login1
o,nPressedl:() {

print(nameController.text); print(passwordController.text);

},

)), II RaisedButton, Container

], II <Widget>[]

72 J

))) ; II ListView, Padding, Scaffold

73 }

:'.i }

import 'package:flutter/material.dart';

void main() { runApp(MaterialApp(home: MyApp(),

));

}

class MyApp extends StatefulWidget { @override

_State createState() => _State();

@override

Widget build(BuildContext context) { return Scaffold(appBar:
AppBar(title: Text('Login'),

),

body: Padding(

padding: EdgeInsets.all(10), child: ListView(

children: <Widget>[

}

class _State extends State<MyApp> {

TextEditingController nameController = TextEditingController(); TextEditingController passwordController = TextEditingController();

Container(
alignment: Alignment.center, padding: EdgeInsets.all(10), child: Text(

'Sign in',

style: TextStyle(fontSize: 20),

. o

)),

Container(

padding: EdgeInsets.all(10), child: TextField(

controller: nameController, decoration: InputDecoration(border: OutlineInputBorder(), labelText: 'User Name',

),

),

),

Container(

padding: EdgeInsets.fromLTRB(10, 10, 10,

0), child: TextField(obscureText: true,

controller: passwordController, decoration: InputDecoration(border: OutlineInputBorder(), labelText: 'Password',

Index

8.3 Button

Elevated Button

Elevated.icon Button

StyleFrom

OutlinedButton

OutlinedButton.icon

TextButton

TextButton.icon

IconButton

Inkwell

Button

Button

A button consists of an icon or a Text(or both).

When the user touches on it, It will trigger the click event and get the appropriate action.

Flutter has various types of buttons. These button types utilized for different purposes.

What are the Button Types in Flutter?

Eleveted Button

Outlined Button

TextButton

Button Types

a) FlatButton — The previously used FlatButton is now known as TextButton and uses the theme of TextButtonTheme.

b) RaisedButton — The previously used RaisedButton is now known as ElevatedButton

and uses the theme of ElevatedButtonTheme.

c) OutlineButton — The previously used OutlineButton is now known as

OutlinedButton(a pretty small change in name here) and uses the OutlinedButtonTheme.

Elevated Button

There are two required parameters. Of course you have to pass a Widget as child, typically a Text or an Icon.

You're also required to pass onPressed callback which is called when the user presses the button.

If you want to create an elevated button with both Icon and Text widgets inside, a static factory method ElevatedButton.icon makes it easier for you.

Instead of child, you need to pass a Widget for each of icon and label.

Basic Usage

```
ElevatedButton( child: Text('Akash'), onPressed: () { print('Pressed');
},

)
```

Output

LongPress

To delect when the user does long press on the button, you can pass another callback function as onLongPress.

```
ElevatedButton(  child: Text('Akash'),
onPressed: () { print('Pressed');
},

onLongPress: () { print("Long Pressed");
},

)
```

Button Style

The style of the button can be customized by creating a ButtonStyle.

The ButtonStyle can be passed as ThemeData's elevatedButtonTheme or ElevatedButton's style.

To pass the ButtonStyle as theme data, you have to create an ElevatedButtonThemeData with the ButtonStyle passed as style parameter in the constructor. Then, pass the ElevatedButtonThemeData as elevatedButtonTheme in the constructor of ThemeData.

Setting the style as theme data affects the style of all ElevatedButton widgets under the tree.

The best practice for creating a ButtonStyle for an ElevatedButton is by using ElevatedButton.styleFrom

Design Button

We can Design Button with 2 Options

ElevatedButtonThemeData

Used for Designing Multiple Button from Single Code.

ElevatedButton.StyleFrom

Used for Designing Individual Button with Different Style.

ElevatedButtonThemeData

ButtonTheme Data for All Button

class _MyAppState extends State<MyApp> { @override Widget build(BuildContext context) { return MaterialApp(

title: 'Flutter', theme:

ThemeData(

elevatedButtonTheme: ElevatedButtonThemeData(style:

ElevatedButton.styleFro m(foregroundColor: Colors.amber, backgroundColor: Colors.black)),

),

home:

Scaffold(appBar:

AppBar(

title: const Text('FlutterApp'),

),

```
body: Center(
child: Column(children: [ ElevatedButton(
        onPressed: () { print("Clicked");
},
child: const Text('Button1')), ElevatedButton(
        onPressed: () { print("Clicked");
},
child: const Text('Button2')),

]),
),
),
);
}
}
```

ElevatedButton.StyleFrom

For defining a style for a specific button, you can pass ButtonStyle as style parameter in the constructor of ElevatedButton.

ElevatedButton.StyleFrom

creating a ButtonStyle for an ElevatedButton is by using ElevatedButton.styleFrom

```
ElevatedButton(
style:    ElevatedButton.styleFrom( foregroundColor:   Colors.white,
backgroundColor:        Colors.black, shadowColor: Colors.red,
),
```

onPressed: () { print("Clicked");

},

child: const Text('Button1')),

Setting Text Style

You can define a TextStyle specific for the button and pass it as textStyle. The color property of TextStyle may not affect the text color, but you can set the color of the text by passing primary as the parameter of ElevatedButton.styleFrom.

ElevatedButton(

style: ElevatedButton.styleFrom(foregroundColor: Colors.white, backgroundColor: Colors.black, shadowColor: Colors.red, textStyle: const TextStyle(

color: Colors.black, fontSize: 30,

fontStyle: FontStyle.italic),

),

onPressed: () { print("Clicked");

},

child: const Text('Button1'))

ElevetedButton Icon

If you want to create an elevated button with both Icon and Text widgets inside, a static factory method ElevatedButton.icon makes it easier for you.

Instead of child, you need to pass a Widget for each of icon and label.

ElevetedButton Icon

ElevatedButton.icon(

onPressed: () { print("Clicked");

},

label: const Text('Call Akash'), icon: const Icon(Icons.call)

),

All Useful Properties

ElevatedButton(

style: ElevatedButton.styleFrom(foregroundColor:

Colors.white, backgroundColor:

Colors.black, shadowColor:

Colors.red, elevation: 5,

side: const BorderSide(color: Colors.yellow, width: 1), textStyle: const TextStyle(

color: Colors.black, fontSize: 15,

fontStyle: FontStyle.italic),

shape: const BeveledRectangleBorder(

borderRadius: BorderRadius.all(Radius.circular(3))),

),

onPressed: () { print("Clicked");

},

child: const Text('Button1')),

ElevatedButton.styleFrom static method.

Color primary: Used to construct ButtonStyle.backgroundColor.

Color onPrimary: Used to construct ButtonStyle.foregroundColor and ButtonStyle.overlayColor.

Color onSurface: Used to construct ButtonStyle.backgroundColor and ButtonStyle.foregroundColor when the button is disabled.

Color shadowColor: The shadow color of the button's Material.

double elevation: The elevation of the button's Material.

TextStyle textStyle: The style for Text widget.

EdgeInsetsGeometry padding: The padding between the button's boundary and its child..

Size minimumSize: The minimum size of the button.

BorderSide side:The color and weight of the button's outline.

OutlinedBorder shape: The shape of the button's underlying Material.

MouseCursor enabledMouseCursor: Defines the MouseCursor when the button is enabled. Used to construct ButtonStyle.mouseCursor.

MouseCursor disabledMouseCursor: Defines the MouseCursor when the button is disabled. Used to construct ButtonStyle.mouseCursor.

VisualDensity visualDensity: How compact the button's layout will be.

MaterialTapTargetSize tapTargetSize: The minimum size of area where the button may be pressed..

Duration animationDuration: The duration of animated changes for shape and elevation.

bool enableFeedback: Whether detected gestures should provide acoustic and/or haptic feedback..

Outlined Button

OutlinedButton

OutlinedButton is a widget that allows you to easily create a Material Design's Outlined Button

There are two required parameters. First, you need to pass a Widget as child, usually a Text or an Icon widget. The other required parameter is onPressed, a callback to be called when the button is pressed.

OutlinedButton(

child: const Text('AkashSir.com'), onPressed: () { print('Pressed');

},

),

OutlineButton.StyleFrom

OutlinedButton.styleFrom

Color primary,: Used to construct ButtonStyle.foregroundColor and ButtonStyle.overlayColor.

Color onSurface: Used to construct ButtonStyle.foregroundColor.

Color backgroundColor: The background color of the button.

Color shadowColor: The shadow color of the button's Material.

double elevation: The elevation of the button's Material.

TextStyle textStyle: The style for Text widget.

EdgeInsetsGeometry padding: The padding between the button's boundary and its child..

Size minimumSize: The minimum size of the button.

BorderSide side:The color and weight of the button's outline.

OutlinedBorder shape: The shape of the button's underlying Material.

MouseCursor enabledMouseCursor: Defines the MouseCursor when the button is enabled. Used to construct ButtonStyle.mouseCursor.

MouseCursor disabledMouseCursor: Defines the MouseCursor when the button is disabled. Used to construct ButtonStyle.mouseCursor.

VisualDensity visualDensity: How compact the button's layout will be.

MaterialTapTargetSize tapTargetSize: The minimum size of area where the button may be pressed..

Duration animationDuration: The duration of animated changes for shape and elevation.

bool enableFeedback: Whether detected gestures should provide acoustic and/or haptic feedback..

OutlinedButton Icon

If the outlined button contains Icon and Text, it can be created using OutlinedButton.icon in which you need to pass a Widget as icon and another one as label instead of passing a Widget as child..

OutlinedButton.icon(

onPressed: () { print('Pressed');

},

label: const Text('Visit Website'), icon: const Icon(Icons.web),

),

TextButton

Text Button is a Material Design's button that comes without border or elevation change by default. Therefore, it relies on the position relative to other widgets.

It can also be used inside a Card widget or dialog and should be grouped in one of the bottom corners. In Flutter, this button can be created using TextButton widget which was introduced in Flutter 1.22

TextButton

TextButton(

child: Text('AkashSir.com'), onPressed: () { print('Pressed');

})

TextButton Icon

TextButton.icon(

label: const Text('AkashSir.com'), icon: const Icon(Icons.email), onPressed: () {

print('Pressed');

})

IconButton

InkWell

InkWell

InkWell class in Flutter is a rectangular area in Flutter of a material that responds to touch in an application.

Tap Down

Double Tap

Long Press

Single Tap

Tap Cancel

InkWell on Container

InkWell(

child: Container(width: 200,

height: 80,

alignment: Alignment.center,

child: const Text('Demo Container'),

),

onTap: () {

// do something

})

Inkwell

InkWell(

splashColor: Colors.yellow, highlightColor: Colors.blue,

child: const Icon(Icons.add_circle, size: 50), onTap: () {

print("onTap Called");

},

),

Inkwell on Text

```
InkWell(
        child: Text("Click Here"), onTap: () =>
        {print('Clicked')},
),
```

```
InkWell( onTap: () {}, child: Ink( width: 200,
height: 200,
color: Colors.blue,
),
)
```

Gesture Detector

Gesture Detector is used to detect the user's gestures on the application. It is a non-visual widget. Inside the gesture detector, another widget is placed and the gesture detector will capture all these events (gestures) and execute the tasks accordingly.

Gestures

```
GestureDetector(
        onTap: () { print("Clicked");
},
child: Container( height: 50,
width: 100,
color: Colors.amber,
child: const Text('Tap Button'),
```

),

),

What are the differences between InkWell and GestureDetector?

They both provide many common features like onTap, onLongPress

etc. The main difference is GestureDetector provides more controls like dragging etc. on the other hand it doesn't include ripple effect tap, which InkWell does.

You can use either of them according to your needs, you want ripple effects go with InkWell, need more controls go with GestureDetector or even combine both of them.

Properties

FloatingAction

FloatingActionButton

A FloatingActionButton in material design is a button on a screen that is tied to an obvious action which a user would usually do on that specific screen.

This button floats above the content of the screen and usually resides on one corner of the screen.

Types of FloatingActionButton in Flutter

FloatingActionButton

FloatingActionButton.extended

The default constructor creates a simple circular FAB with a child widget inside it.

FloatingActionButton

It takes an onPressed method to react to taps and a child (not compulsory) to display a widget inside the FAB.

FloatingActionButton(onPressed: () {}, child: Icon(Icons.add),

),

FloatingActionButton.extended

FloatingActionButton.extended offers a wide FAB, usually with aniconandalabelinsideit.

FloatingActionButton.extended(onPressed: () {}, icon: Icon(Icons.save), label:

Text("Save"),
),

FloatingActionButton

class MyApp extends StatelessWidget { @override

Widget build(BuildContext context) {

return new MaterialApp(

L title: "MyApp",

home: new Scaffold(appBar: new AppBar(

Ltitle: new Text("Akash Padhiyar"),

), // AppBar

body: new Center(child: new Text("Hello World",

style: TextStyle(fontSize: 50.0))), // Text, Center floatingActionButton: new FloatingActionButton(Lchild: Text('Hi'), //chiLd: new Icon(Icons.add_caLL),

backgroundColor: Colors.redAccent,

onPressed: (){ print('Yes');},

), // FLoatingActionButton

), // ScaffoLd

theme: new ThemeData(

primarySwatch: Colors.deepOrange,

brightness: Brightness.light //dark Light

), // ThemeData

); // MateriaLApp

}

}

Hello World

floatingActionButton: FloatingActionButton(child: Icon(Icons.chat), backgroundColor: Colors.green, foregroundColor: Colors.white,

tooltip: 'Upload', onPressed: () => {},

),

FloatinActionButton.extended

5

6

7 of

8

9

class MyApp extends StatelessWidget { @override

Widget build(BuildContext context) {

return new MaterialApp(

L title: "MyApp",

home: new Scaffold(appBar: new AppBar(

Ltitle: new Text("Akash Padhiyar"),

), // AppBar

body: new Center(child: new Text("Hello World",

style: TextStyle(fontSize: 50.0))), // Text Center floatingActionButton: new FloatingActionButton.extended(ticon: new Icon(Icons.call),

label: new Text('Help'), backgroundColor: Colors.redAccent, foregroundColor: Colors.yellow, onPressed: (){ print('Yes');},

), // FloatingActionButton.extended

), // Scaffold

theme: new ThemeData(

primBrySwatch: Colors.deepOrange,

brightness: Brightness.light //dark light

), // ThemeData

); // MaterialApp

} }

floatingActionButton: FloatingActionButton.extended(label: Text('Upload'),

icon: Icon(Icons.chat), backgroundColor: Colors.green, foregroundColor: Colors.white, tooltip: 'Upload',

onPressed: () => {},

)

Load image from the Assets

It is a widget that displays an image.

8.4 Image Class

The Flutter app supports many image formats, such as JPEG, WebP, PNG, GIF, animated WebP/GIF, BMP, and WBMP.

Constructors

Image()

Creates a widget that displays an image.

Options

Image.asset()

Creates a widget that displays an ImageStream obtained from an asset bundle. The key for the image is given by the name argument.

Image.network()

Creates a widget that displays an ImageStream obtained from the network.

Image.file()

Creates a widget that displays an ImageStream obtained from a File.

Image.memory()

Creates a widget that displays an ImageStream obtained from a Uint8List.

Properties

Height
If non-null, require the image to have this height.

Image
The image to display.

Width
If non-null, require the image to have this width.

Key
Controls how one widget replaces another widget in the tree.

Alignment
How to align the image within its bounds.

centerSlice

The center slice for a nine-patch image.

Color
If non-null, this color is blended with each image pixel using colorBlendMode.

colorBlendMode

Used to combine color with this image.

excludeFromSemantics

Whether to exclude this image from semantics.

filterQuality

Used to set the FilterQuality of the image.

Fit
How to inscribe the image into the space allocated during layout.

frameBuilder

A builder function responsible for creating the widget that represents this image.

ImageAsstes

To begin, create an assets folder.

ImageAssets

The assets folder should be located in the same directory as the pubsec.yaml file in your project.

Once the image has been added to the assets folder, the pubsec.yaml file needs to be edited to incorporate the image.

Any external resource or package in your program must similarly be included in the pubsec.yaml file.

Steps

Create New Folder in Root Directory

assets

images

akash.jpeg

Register Folder in pubspec.yaml File. assets:

- assets/images/

Use ImageAsset Widget

Image.asset('assets/images/akash.jpeg'),

Create Folder

Create a new directory called assets/images.

Add Image in Folder

Adding Image Assets to a Flutter Project

2Space–1Space

Adding Image Assets to a Flutter Project

Basic Structure

import 'package:flutter/material.dart'; void main() {

runApp(const MyApp());

}

class MyApp extends StatefulWidget { const MyApp({super.key}); @override

State<MyApp> createState() => _MyAppState();

}

class _MyAppState extends State<MyApp> { @override Widget build(BuildContext context) { return MaterialApp(

---

title: 'Flutter', home:

Scaffold(appBar:

AppBar(

title: const Text('FlutterApp'),

),

body: Column(children: const [Text(

"Image Demo", textAlign: TextAlign.center, style: TextStyle(

fontSize: 20,

),

)

]),

);

}

}

Image assets

import 'package:flutter/material.dart'

; void main() { runApp(const MyApp());

}

class MyApp extends StatefulWidget { const MyApp({super.key}); @override

State<MyApp> createState() => _MyAppState();

}

class _MyAppState extends State<MyApp> { @override

Widget build(BuildContext context) { return MaterialApp(

title: 'Flutter', home:

Scaffold(appBar:

AppBar(

title: const Text('FlutterApp'),

),

body: Column(children: [const Text("Image Demo", textAlign: TextAlign.center, style: TextStyle(

fontSize: 20,

),

Image.asset('assets/images/akash.jpeg'),

]),

),

);

}

}

Image.asset(

'assets/images/akash.jpeg', width: 250,

height: 250,

),

Image in Container

import 'package:flutter/material.dart'; void main() {

runApp(const MyApp());

}

class MyApp extends StatefulWidget { const MyApp({super.key}); @override

State<MyApp> createState() => _MyAppState();

}

class _MyAppState extends State<MyApp>

{ @override

Widget build(BuildContext context) { return MaterialApp(title: 'Flutter',

home:

Scaffold(appBar:
AppBar(

title: const Text('FlutterApp'),

),

body: Container(color:
Colors.red[200], width: 400,

height: 500,

alignment: Alignment.center,

child: Image.asset('assets/images/akash.jpeg'),

),

),

);

}

}

Properties

Height:100

Width:100

Other Parameter

alignment: Alignment.bottomCenter

fit: BoxFit.contain

repeat: ImageRepeat.repeat

scale: 2.5,

Repeat Example

Repeat x y Container(

color: Colors.yellow, width: 100, height: 100,

child: Image.asset("assets/images/call.png", repeat: ImageRepeat.repeat,

),

),

Opacity Widget

import 'package:flutter/material.dart'; void main() {

runApp(const MyApp());

}

class MyApp extends StatefulWidget { const MyApp({super.key}); @override

State<MyApp> createState() => _MyAppState();

}

class _MyAppState extends State<MyApp> { @override

Widget build(BuildContext context)

{ return MaterialApp(title: 'Flutter',

home: Scaffold(appBar: AppBar(

title: const Text('FlutterApp'),

),

body: Opacity(opacity: 0.5,

child: Image.asset('assets/images/akash.jpeg'),

),

),

);

}

}

Command

flutterpub get

flutterclean

Network Image

Network Image will help to Load Image from Online Resource rather than storing in app.

Internet Connection will required to load application.

Flaticon.com

Get Image From Online

Copy Image URl

Image Network

import 'package:flutter/materi al.dart'; void main() { runApp(const MyApp());

}

class MyApp extends StatefulWidget { const MyApp({super.key})

; @override

State<MyApp> createState() =>

_MyAppState();

}

class _MyAppState extends State<MyApp> { @override

Widget build(BuildContext context) { return

MaterialAp p(title:

'Flutter', home:

Scaffold(appBar:

AppBar(

title: const Text('FlutterApp'),

),

body: Container(color:

Colors.grey[200

], child: Image.network('https://cdn-icons-

png.flaticon.com/512/733/733547.png',

),

),

),

);

}

}

Image File

To load images from the file system in the target device, you must use Image.file. However, you must first ensure that the app has the proper permissions to access the device's external storage.

We need to Check Permission in AndroidManifest.xml

AndroidManifest.xml

<manifest xmlns:android package= "flutter.image.demo">

<uses-permission android:name = "android.permission.READ_EXTERNAL_STORAGE"/>

<application

...

</application>

</manifest>

Android/app/src/main/AndroidManifest.xml

Location

We need to Import Dart:io

Get File Path from Device.

```
import 'dart:io';
```

Code

Image.file(

new File('/storage/emulated/0/Download/forest.jpg')),

alignment: Alignment.center,

)

Text Over Image

The Stack the widget allows us to put up multiple layers of widgets onto the screen.

Stack Widget

The widget takes multiple children and orders them from bottom to top. So the first item is the bottommost and the last is the topmost.

Text Over Image

CircleAvatar

CircleAvatar(

backgroundColor: Colors.grey.shade300, child: Text('Y'),

)

Circle Avtar with Network Image

CircleAvatar(

),

backgroundImage: NetworkImage(

"https://cdn-icons-png.flaticon.com/512/733/733547.png"),

Icon

Icon

Icon class in Flutter is used to show specific icons in our app. Instead of creating an image for our icon, we can simply use the Icon class for inserting an icon in our app.

For using this class you must ensure that you have set uses-material-design: true in the pubsec.yml file of your object.

Properites

color: It is used to set the color of the icon

size: It is used to resize the Icon

semanticLable: It comes in play while using the app in accessibility mode (ie, voice-over)

textDirection: It is used for rendering Icon

icon

Icon(

Icons.favorite, color: Colors.red, size: 300,

),

Radio

Flutter Radio widget is used to display a material design radio button.

Radio

Properties

value is the value represented by this radio button.

groupValue is the value that is currently selected for this group of radiobuttons.

onChanged is the callback function that is called when user selects thisradiobutton.

RadioListTile(

value: "Male", groupValue:

_result, selected: true, title: const Text("Male"),

onChanged: (value) => { setState(() {

_result = "Male";

})

}),

Android Emulator - flutter_emulator:5554

Code

var _result = "Male"; Column(

children: [RadioListTile(

value: "Male", groupValue: _result, selected: true,

title: const Text("Male"), onChanged: (value) =>

{

setState(() {

_result = "Male";

})

}),

RadioListTile(value: "Female",

groupValue: _result, title: const Text("Female"), onChanged: (value) => {

setState(() {

_result = "Female";

})

}),

Text(_result == "Male" ? "Male" : "Female"),

],

),

8.5 Checkbox

In Flutter, we can implement checkboxes by using the Checkbox widget or the CheckboxListTile widget.

var _result = true; CheckboxListTile(

value: _result,

title: const Text("Accept the Terms?"), onChanged: (value) => {

setState(() {

_result = !_result;

})

}),

Text("$_result"),

Android

iOS

[iOS, Android)

8.6 Visibility Widget

The Visibility widget is used to show and hide the widget. The Visibility widget has a property called visible.

It accepts a boolean value. Setting the True will show the widget while setting it to false will hide the widget.

Code

Visibility(

visible: true, child: Text("Hello world!",

style: TextStyle(fontSize: 30),

),

),

Visible True False

> ***class _MyAppState extends State<MyApp> { @override Widget build(BuildContext context) { return Scaffold(***

appBar: AppBar(

title: const Text('FlutterApp'),

),

body: Column(children:

const [Visibility(visible:

false, child:

Text("Hello world!",

style: TextStyle(fontSize: 30),

),

),

]),

);

}

}

Maintaining the Size when Hidden

by default the Visibility widget removes the child widget from the widget tree.

Hence the space reserved by the child widget gets free and the Text gets shifted. In some cases, this can annoy your users as all the UI below gets shifted.

To fix this issue, we can preserve the space (reserved by the child widget) by setting the maintain flags to true.

Add the maintainSize, maintainAnimation, maintainState parameter, and set its value to true.

Default Size will be Hidden

Size Maintained

Setting the combination of maintainSize, maintainAnimation, maintainState to true will keep the space as it is and the UI below the hidden widget won't be shifted.

Visibility(

visible: false, maintainSize: true, maintainAnimation: true, maintainState: true, child: Text(

"Second Line!",

style: TextStyle(fontSize: 30),

),

)

Replacement Widget

Sometimes you may want to display another widget when the primary widget is hidden.

Visibility(

visible: true,

replacement: Text("Replacement", style: TextStyle(fontSize: 30)), child:

Text("Visible Widget ",

style: TextStyle(fontSize: 30),

),

),

Replacement Widget

Toggel Visible

class _MyAppState extends State<MyApp> { bool is_visible = true; @override

---

Widget build(BuildContext context) { return Scaffold(

appBar: AppBar(

title: const Text('FlutterApp'),

),

body: Column(child ren: [

Visibility(visible: is_visible, child:

const Text("Hello world!",

style: TextStyle(fontSize: 30),

),

),

ElevatedBu tton(onPressed

: (() {

setState(() {

is_visible = !is_visible;

});

}),

child: Text(is_visible ? "Hide" : "Show")),

]),

);

}

}

8.7 Navigation Drawer

The navigation drawer is useful when you want to perform different page actions and events on different pages with the use of switching pages in the main drawer page.

Header

R Houses

Navigation { Apartments

Links

Section {

Divider •

Townhomes Favorites

child: We use ListView in this field because generally we place a lot of options here.

backgroundColor: Colour for the background of Drawer widget.

shape: If you want different shape of Drawer rather than rectangle, this option is for you.

elevation: The Drawer widget has an elevation over current layer. So there is an shadow to the container. This option helps to change the default value of 16.0.

semanticLabel: Allows frameworks know when the drawer is opened or closed.

Basic App

Drawer Option

To add NavigationDrawer optionadd belowcode

return Scaffold(appBar: AppBar(

title: Text("Drawer app"),

),

drawer: Drawer(),//this will just add the Navigation Drawer Icon

),

Navigation Drwaer

add a ListView as a child of Drawer Widget

drawer: Drawer(child: ListView(

children: <Widget>[],

),

),

dd as many as Widgets inside the listView

drawer: Drawer(

child: ListView(children: const [ListTile(

title: Text("Menu1"),

trailing: Icon(Icons.arrow_forward),

),

ListTile(

title: Text("Menu2"),

trailing: Icon(Icons.arrow_forward),

),

]),

),

Menu

endDrawer and the drawer will be open from the right

Navigation Drawer in right side

Navigation Drawer in right side (endDrawer)

Header

drawer: Drawer(

child: ListView(children: const [DrawerHeader(child: Text("Header"),

),

ListTile(

title: Text("Menu1"),

trailing: Icon(Icons.arrow_forward),

),

]),

),

Header

Menul

UserAccountsDrawerHeader

Code

drawer: Drawer(

child: ListView(children: const [UserAccountsDrawerHeader(currentAccountPicture: CircleAvatar(

backgroundImage: NetworkImage('https://akashtechnolabs.com/upload/images/about/AboutCEO.jpg'),

),

accountEmail: accountName: Text(

'Akash Padhiyar',

style: TextStyle(fontSize: 24.0),

),

arrowColor: Colors.white, decoration: BoxDecoration(color: Colors.black87,

),

),

ListTile(

title: Text("Menu1"),

trailing: Icon(Icons.arrow_forward),

),

]),

),

drawer: Drawer(

child: ListView(children: const [UserAccountsDrawerHeader(currentAccountPicture: CircleAvatar(

backgroundImage: NetworkImage('https://akashtechnolabs.com/upload/images/about/AboutCEO.jpg'),

),

accountEmail: accountName: Text(

'Akash Padhiyar',

style: TextStyle(fontSize: 24.0),

),

arrowColor: Colors.white, decoration: BoxDecoration(color: Colors.black87,

),

),

ListTile(

leading: Icon(Icons.home), title: Text("Home"),

),

ListTile(

leading: Icon(Icons.account_box), title: Text("About"),

),

ListTile(

leading: Icon(Icons.contact_mail), title: Text("Contact"),

)

]),

),

TextField State

TextField OnChange

TextField has an onChanged parameter for a callback method.

This method provides the current string after a change has been made.

In example whenever a TextField is changed, the Text widget above it gets updated.

onChange Text

8.7 On Change Example

class _MyScreenState extends State<MyScreen> { String _textString = "";
@override

Widget build(BuildContext context) { return MaterialApp(

title: 'Flutter Demo', home: Scaffold(

appBar: AppBar(

title: const Text('Flutter Demo'),

),

body: Column(children: [const Text(

'Flutter App',
style: TextStyle(fontSize: 25),

),

const TextField(

decoration: InputDecoration(labelText: 'Enter your name',

),

),

Text(
_textString,

),

],

)),

);

}

}

On Change Text Update

TextField(

onChanged: ((value) => { setState(() {

myname = value;

})

}),

),

Code

class _MyScreenState extends State<MyScreen> { String _textString = "";

@override

Widget build(BuildContext context) { return MaterialApp(title: 'Flutter Demo',

home: Scaffold(appBar: AppBar(

title: const Text('Flutter Demo'),

),

body:

Column(children: [const Text(

'Flutter App',

style: TextStyle(fontSize: 25),

),

TextField(decoration: const

InputDecoration(labelText: 'Enter your name',

),

onChanged: (text) {

_changeContent(text);

},

),

Text(

_textString,

),

],

)),

changeContent(String text) { setState(() {

_textString = text;

});

}

}

);

}

Text Value

Text On Button Click

Basic Design Structure

On Button Click Print name on Text

class _MyScreenState extends State<MyScreen> { @override Widget build(BuildContext context) { return MaterialApp(

---

title: 'Flutter Demo', home: Scaffold(

appBar: AppBar(

title: const Text('Flutter Demo'),

),

body:

Column(children: [const Text(

'Flutter App',

style: TextStyle(fontSize: 25),

),

const TextField(

decoration: InputDecoration(labelText: 'Enter your name',

),

),

ElevatedButton(onPressed: () {

print("Click Event Fired");

},

child: const Text('Click me')), const Text(

'Hello World',

),

],

)),

);

}

}

On Submit Value Print

_changeContent(String text) { setState(() {

_tempString = text;

});

}

_showContent(String text) { setState(() {

_textString = _tempString

});

}

State and Button Click

Controller

Controller Example

Code

class _MyScreenState extends State<MyScreen> {
TextEditingController txt1 = TextEditingController(); String myText

= "";

@override

Widget build(BuildContext context) { return MaterialApp(title: 'Flutter

Demo', home:

Scaffold(

appBar: AppBar(

title: const Text('Flutter Demo'),

),

body:

Column(children: [const Text(

'Flutter App',

style: TextStyle(fontSize: 25),

),

TextField(decoration: const InputDecoration(

labelText: 'Enter your name',

),

controller: txt1,

),

```
ElevatedBut ton( onPressed: () {
_showContent();
},
child: const Text('Click me')), Text( myText,
style: const TextStyle(fontSize: 25),
),
],
)),
```

showContent() { print("Text is

```
v   ${txt1.text}"); setState(() {
o       myText = txt1.text;
i       });
d       }
_       }
);
}
```

Code Snippets

```
TextEditingController txt1 = TextEditingController(); String ans = "";
```

TextField(

controller: txt1,

),

setState(() {

ans = txt1.text;

});

Year Leap Program

void myprocess() {

var a = int.parse(txt1.text);

if (a % 2 == 0) {

setState(() { ans = "Even";

});

} else { setState(() { ans = "Odd";

});

}

}

Leap Year

class _MyScreenState extends State<MyScreen> {
TextEditingController txt1 = TextEditingController();
TextEditingController txt2 = TextEditingController(); String myText = "";

@override Widget

build(BuildContext context) {

return MaterialApp(title: 'Flutter Demo', home: Scaffold(

appBar: AppBar(title: const Text('Flutter Demo'),

),

body: Column(children:

[const Text(

'Sum Example', style:

TextStyle(fontSize: 25),

),

TextField(

decoration: const InputDecoration(labelText: 'Enter No 1',

),

controller: txt1,

),

TextField(

decoration: const InputDecoration(labelText: 'Enter No 2',

),

controller: txt2,

),

ElevatedButton(onPressed: () {

_showContent();

},

child: const Text('Click me')), Text(myText,

style: const TextStyle(fontSize: 25),

),

],

)),

void _showContent() { v

a

);

}

r no1 = int.parse(txt1.text); var no2 = int.parse(txt2.text); var c = no1 + no2; setState(() {

myText = "Sum is ${c}";

});

}

}

Task

Create Mini Calculator having 4 Buttons + - * / of 2 Numbers.

Number is Even or Odd Check

Take 5 Subject from user and Print all Marks , % Total

Basic 4-5 Program

8.9 Navigation in Flutter

Navigator class

Mobile apps typically reveal their contents via full-screen elements called “screens” or “pages.”

In Flutter, the screen and pages are called a route

Android : Activity

iOS : ViewController

Mobile App : Screens

Website : Pages

In Flutter, these elements are called routes, and they’re managed by a Navigator widget.

The navigator leads a stack of Route objects and provides methods for controlling the stack.

Flutter provides the routing class MaterialPageRoute, and two methods Navigator.push() and Navigator.pop() to handle navigations.

Methods

The Navigator class provides all the navigation capabilities in a Flutter app.

Navigator.push = Add Screen

Navigator.pop. = Remove Screen

Navigator.push

It pushes the given route onto the navigator that most tightly encloses the given context.

Syntax:-

```
Navigator.push(

context,

    MaterialPageRoute(builder: (context)  =>  SecondRoute()),

);
```

Navigator.pop

It pops the top-most route off the navigator that most tightly encloses the given context.

Syntax:-

```
Navigator.pop(context);
```

```
import 'package:flutter/material.dart';


void main() => runApp(const MaterialApp( title: "App", home:
MyApp(),
```

FirstRoute

```
class MyApp extends StatelessWidget { const MyApp({super.key});
@override
```

Widget build(BuildContext context) { return Scaffold(

appBar: AppBar(

title: const Text('Home Route'),

),

body: Center(

child: ElevatedButton(

child: const Text("Click to Navigate"), onPressed: () {

Navigator.push(context,

MaterialPageRoute(builder: (context) => const SecondRoute()),

);

},

),

));

}

}

SecondRoute

class SecondRoute extends StatelessWidget { const SecondRoute({super.key}); @override Widget build(BuildContext context) { return Scaffold(

appBar: AppBar(

title: const Text('Second Route'), backgroundColor: Colors.red,

),

body: Center(

child: ElevatedButton(

style: ElevatedButton.styleFrom(backgroundColor: Colors.red, // background foregroundColor: Colors.white, // foreground

),

child: const Text("Back"), onPressed: () { Navigator.pop(context);

}),

),

);

}

}

Second and Third Route

Navigate with Named Routes in Flutter (pushNamed)

Navigate with Named Routes in Flutter

Named Routes is the simplest way to navigate to the different screens using naming concepts.

When a user needs multiple screens in an app depending on its need then we have to navigate from one screen to another and also return back many times back on the previous screen then it becomes a very hectic task,

there we use a naming concept so as to remember screens with its name provided by the user and then we can directly access that route or page directly through Navigator.pushNamed() method

Defining the routes with names

pushNamed

import 'package:flutter/material.dart'; void main()

=> runApp(MaterialApp(title: "App",

home: MyApp(), initialRoute:

"Home", routes: {

"Home": (context) => const MyApp(), "Second": (context) => const SecondRoute(), "Third": (context) => const ThirdRoute(),

},

));

3 Routes and Names Navigation

Navigator.pushNamed(context, 'Second');

Navigator.pushNamed(context, 'Third');

Navigator.pushNamed(context, '/');

pushReplacementNamed

In pushReplacementNamed, the current route of the navigator pushes the route named [routeName] and then dispose of the previous route once the new route has finished animating

Example

When the user has logged in successfully, and are now on the DashboardScreen perhaps, then you don't want the user to go back to LoginScreen in any case. So login route should be completely replaced by the dashboard route.

Another example would be going to HomeScreen from SplashScreen. It should only display once and the user should not be able to go back to it from HomeScreen again. In such cases, since we are going to a completely new screen, we may want to use this method for its enter animation property.

pushReplacementNamed

Navigator.pushReplacementNamed(context, 'Third');

No Back Button

All

popAndPushNamed

.pop()

to return to the previous page.

.popUntil((route) => condition)

to navigate back to the previous page until the desired site appears.

.pushReplacement(newRoute)

to replace a page with the current one.

.popAndPushNamed(routeName)

is similar to the previous one, but before that, the navigation to the page is made uniquely identifiable with a string.

.pushNamed(routeName)

is similar to the previous one, but without replacing the current page with it.

Task

Create new Flutter Project take 3 Butttons and onclick event navigate on different Screen.

Create New Project having 2 Different Screen

Signup (4 Textfield)

Login (2 Textfield)

Also Specify navigation between 2 screen.

Nested Navigation

Screen 1 on Button click Navigate on Screen 2

Screen 2 on Button Click Navigate on Screen 3

Shopping Cart App Screen List

Splash Screen

Login

Signup

CategoryList

ProductList

Details

AddtoCart

PlaceOrder

ViewOrder

AboutUs

Contact

Music

A Flutter plugin to play multiple simultaneously audio files, works for Android, iOS, Linux, macOS, Windows, and web.

https://pub.dev/packages/audioplayers

audioplayN"slF1uttj"rPacka¥ X +

f- "7 C O i pub.dev/packages/audioplayers/install

Use this package as a library

Depend on it

Run this command:

With Flutter:

$ flutter pubaddaudioplayers

This will add a line like this to your package's pubspec.yaml (and run an implicit flutter pubget):

dependencies:

audioplayers: '1. 1.1

Alternatively, your editor might support flutter pubget. Check the docs for your editor to learn more.

Import it

Now in your Dart code, you can use:

import 'package :audioplayers/audioplayers. dart' ;

\„

lib> \,·main.dart > 't:$ _MyHomePageState > 6;I build

84 ··········//·children·horizontally,·and·tr1es·to b

85 •••••••••·II .. .

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL COMMENTS

PROBLEMS OIJTPlJT DEBUG CONSOLE TERMINAL COMMENTS

Add Folder

Import

import 'package:audioplayers/audioplayers.dart';

String myAudioFilePath = "assets/audio/mysong.mp3";

AudioPlayer audioPlayer = AudioPlayer();

Using One Button

8.10 Switch

Flutter Switch widget can be used to ON or OFF a single setting.

Switch has two required attributes: value and onSwitchChange

When user presses on the Switch, onChanged() callback method will be called with the new state value as a boolean value. The boolean value is true if the switch is turned on, or false if the switch is turned off.

OnOff

Properties

Code

bool is_onoff = true; Switch(value: is_onoff,

activeColor: Colors.green, activeTrackColor: Colors.blue, inactiveThumbColor: Colors.amber, inactiveTrackColor: Colors.red, onChanged: (value) {

setState(() { is_onoff = value;

});

},

),

Slider Input

A Slider is an input widget where user can set a value by dragging to or pressing on desired position.

There are only two required arguments: value and onChanged

> ***A Slider widget is usually used to change a value. So, we need to store the value in a variable. The Slider class requires you to implement onChanged function which will be called every time the user changes the slider position.***

The most common thing to do inside the callback is updating the value. The value itself must be passed as value argument, which can be used to set initial value.

Slider

double _myvalue = 1; Slider(min: 1,

max: 10,

value: _myvalue, onChanged: (double value)

setState(() {

_myvalue = value;

});

}),

Other properties

If we want to divide the slider into equal parts we will use divisions property.

Divisons

Code

double _myvalue = 1

Slider(

min: 1,

max: 10,

activeColor: Colors.green, thumbColor: Colors.red, value: _myvalue, onChanged: (double value) { setState(() {

_myvalue = value;

});

},

onChangeStart: (value) => print("Start Dragging..."), onChangeEnd: (value) => print("End Dragging...")),

Properties

Range Slider

Code

double _startValue = 20.0; double _endValue = 90.0;

RangeSlider(

min: 0.0,

max: 100.0,

values: RangeValues(_startValue, _endValue), onChanged: (value) { setState(() {

_startValue = value.start;

_endValue = value.end;

});

}),

Text('$_startValue'), Text('$_endValue'),

CupertinoActivityIndicator

CupertinoActivityIndicator is the Cupertino version of the material circular progress indicator. It animates in a clockwise circle. Flutter possesses in its store a widget to perform this task with ease & perfection Using CupertinoActivityIndicator.

CupertinoActivityIndicator(animating: true, radius: 30,

)

OnClick Show Hide

Show/Hide Widget In Flutter

The Visibility widget is used to show and hide the widget. The Visibility widget has a property called visible. It accepts a boolean value. Setting the True will show the widget while setting it to false willhidethewidget.

Visibility(visible:

true,

child: Image.asset('assets/images/eye_open.png', height: 300,

width: 300,

),

)

Show Hide

Toggel

class _MyAppState extends State<MyApp> { bool is_visible = true;

@override

Widget build(BuildContext context) { return Scaffold(

appBar: AppBar(

title: const Text('FlutterApp'),

),

body: Column(children: [Visibility(

visible: is_visible, child: const Text("Hello world!",

```
style: TextStyle(fontSize: 30),
),
),
ElevatedButton( onPressed: (() {
setState(() {
is_visible = !is_visible;
});
}),
child: Text(is_visible ? "Hide" : "Show")),
]),
);
}
}
```

Chapter - 9 Local Data Storing

9.1 Shared Preferences

What is SharedPreferences?

SharedPreferences is used for storing data key-value pair in the Android and iOS.

SharedPreferences in flutter

UserDefaults on iOS

SharedPreferences on Android,

LocalStorage on Browser

providing a persistent store (Store Data) for simple data.

Storage location by platform

Local Storage Example in Browser

G Google X +

+- C Q i google.com 1B

Google

Gmail

Application

Elements

Console Sources Network Performance Memory Application
Security Lighthouse »

C Filter

I Manifest

(I Service Workers Storage

Storage

,. 55 Local Storage

55

55 https://ogs.google.com

!!!! c;pc;c;inn Stnr;=inp

Key sb_wiz.ueh sb_wiz.zpc.

Value

eda8d98767408d93fd7fbbc736cfe3481ef7a975

{"0":[["videos",0,[362,308,345,357],{"zl":40009}],["cool photos",O,[362,308,345,357],{"zl":40009}] p:*ll:9007199254740991_205

How to View Data in Mobile ?

In Mobile we can not open or view Shared Preference Data.

Why to use SharedPreferences in Flutter?

Suppose we want to save a small value (a flag probably) that we want to refer later sometime when a user launches the application. Then shared preference comes into action.

Example :

Store High Score of Game

Stores Users Favorite Color

Use Preference (Setting : Color,Theme,FontSize,DarkMode)

User Login Auth (isUser Logged in ?)

App Login Details Remember

We can Post SharePrefrence Datato API.

How to use SharedPreferences in Flutter?

Before using SharedPreferences, you should know that Flutter SDK does not have support SharedPreferences but fortunately, the shared_preferences plugin can be used to persist key-value data on disk.

Code

https://pub.dev/packages/shared_preferences

'- s.hared_preferences I Rutt Paci X +

-E- C O i pub.dev/packages/.s.hared_preference:s:/install

Use this package as a library

Depend on it

Run this command:

With Flutter:
$ flutter pub add shared_preferences

This will add a line like this to your package's pubspec.yaml (and run an implicit flutter pub get):

dependencies:
shared_preferences: '2.0.15

Alternatively, your editor might support flutter pub get . Check the docs for your editor to learn more.

Import it

Now in your Dart code, you can use:

```
import 'package:shared_preferences/shared_preferences.dart';
```

Steps

Step 1: Add the dependencies in pubspec.yaml

shared_preferences: ^2.0.15

Step 2 : Import plugins

```
import 'package:shared_preferences/shared_preferences.dart';
```

Get Instance
Obtain shared preferences.

final prefs = await SharedPreferences.getInstance();

Quick Code
Obtain shared preferences.

final prefs = await SharedPreferences.getInstance();

Set

```
await prefs.setInt('counter', 10);
```

Get

```
final int? counter = prefs.getInt('counter');
```

Remove

```
final success = await prefs.remove('counter');
```

Set/Write Method

Obtain shared preferences.

```
final prefs = await SharedPreferences.getInstance();
```

Save an integer value to 'counter' key.

```
await prefs.setInt('counter', 10);
```

Save an boolean value to 'repeat' key.

```
await prefs.setBool('repeat', true);
```

Save an double value to 'decimal' key.

```
await prefs.setDouble('decimal', 1.5);
```

Save an String value to 'action' key.

```
await prefs.setString('action', 'Start');
```

Save an list of strings to 'items' key.

```
await prefs.setStringList('items', <String>['Earth', 'Moon', 'Sun']);
```

Read data

Try reading data from the 'counter' key. If it doesn't exist, returns null.

```
final int? counter = prefs.getInt('counter');
```

Try reading data from the 'repeat' key. If it doesn't exist, returns null.

```
final bool? repeat = prefs.getBool('repeat');
```

Try reading data from the 'decimal' key. If it doesn't exist, returns null.

```
final double? decimal = prefs.getDouble('decimal');
```

Try reading data from the 'action' key. If it doesn't exist, returns null.

final String? action = prefs.getString('action');

Try reading data from the 'items' key. If it doesn't exist, returns null.

final List<String>? items = prefs.getStringList('items');

Remove an entry

Remove data for the 'counter' key.

final success = await prefs.remove('counter');

Add the Dependencies in pubspec.yaml

Add the dependencies in pubspec.yaml

Import plugins

import 'package:shared_preferences/shared_preferences.dart';

Widget

Button

Shared Preferences Save Get Remove

var prefs = await SharedPreferences.getInstance(); await prefs.setInt('counter', 10);

final int? counter = prefs.getInt('counter');

var prefs = await SharedPreferences.getInstance();

Save Get Remove

Using TextField

OnLoad Check GetData();

TextEditingController txt1 = TextEditingController(); var myvalue = ""; @override

void initState() { super.initState(); getData();

}

TextField,Save,Get,Remove Button

Center(

child:

```
Column( children: [ TextField(

controller: txt1,

),

Text(myvalue), ElevatedButton(

onPressed: () { saveData();

},

child: const Text("Save")), ElevatedButton(

onPressed: () { getData();

},

child: const Text("Get")), ElevatedButton(

onPressed: () { removeData();

},

child: const Text("Remove")),

],

),

),
```

SaveData GetData RemoveData Functions

```
void saveData() async {

var pref = await SharedPreferences.getInstance(); await pref.setString("mya", txt1.text);

setState(() {

myvalue = "Data Saved";

});

}
```

```
void getData() async {

var pref = await SharedPreferences.getInstance(); var mydata = pref.getString("mya");

if (mydata == null) { setState(() {

myvalue = "No Data Found";

});

} else { setState(() {

myvalue = mydata;

});

}

}

void removeData() async {

var pref = await SharedPreferences.getInstance(); pref.remove("mya");

setState(() {

myvalue = "Removed";

});

}
```

Output

```
CounterApp

void _SaveData() async {

var prefs = await SharedPreferences.getInstance(); await prefs.setInt('counter', 10);

setState(() { myText = "Saved";

});

}

void _GetData() async {

var prefs = await SharedPreferences.getInstance(); final int? counter =
prefs.getInt('counter');
```

```
setState(() {
myText = "Get Data : $counter";
});
}
void _RemoveData() async {
var prefs = await SharedPreferences.getInstance(); await prefs.remove('counter');
setState(() {
myText = "Removed";
});
}
```

Conter

```
int _counter = 0;
void load_Counter() async { SharedPreferences pref = await
SharedPreferences.getInstance(); setState(() {
_counter = (pref.getInt('counter') ?? 0);
});
}
@override
void initState() { super.initState();
return MaterialApp( home: Scaffold(
```

appBar: AppBar(title: const Text("flutter demo")), body: Center(child:

Column(children:

```
[
const Text("you push time:"), Text('$_counter'),
],
),
```

),

floatingActionButton: FloatingActionButton(onPressed: () {

_incrementCounter();

},

child: const Icon(Icons.add),

),

),

load_Counter();

}

void _incrementCounter() async {);

SharedPreferences prefs = await SharedPreferences.getInstance(); setState(() {

_counter = ((prefs.getInt('counter') ?? 0) + 1); prefs.setInt('counter', _counter);

});

}

Counter App Store Current Number in SharePre.

Task

Take a Name From user and Store Name in Preference. On Second Button Click Get name from Pref and Print on Label.

Take a number from User and Store in Pref and onloading of app display TextSize same as Pref Value.

Take Slider and on Change of Slider Increase or Decrees font size

Take 3 Button Red Green Blue on button click Set color name in Preference on load of app display bgcolor according to preference.

Todo App Add Display Remove

Create Signup Screen and Store Variaous Data in Sharedpref. Like Name,Email,Passworda and Mobile No.

Login Screen Tame Email and Password from user and Check Same Email and password is present or not in Pref. and Print Appropriate Message.